

Full Stack AI DevOps for Software Developers (AWS, Azure, GCP)

Faisal Memon (EmbarkX)

Instructor: Faisal Memon

Company: EmbarkX.com

1. Personal Use Only

The materials provided in this course, including but not limited to PDF presentations, are intended for your personal use only. They are to be used solely for the purpose of learning and completing this course.

2. No Unauthorized Sharing or Distribution

You are not permitted to share, distribute, or publicly post any course materials on any websites, social media platforms, or other public forums without prior written consent from the instructor.

3. Intellectual Property

All course materials are protected by copyright laws and are the intellectual property of Faisal Memon and EmbarkX. Unauthorized use, reproduction, or distribution of these materials is strictly prohibited.

4. Reporting Violations

If you become aware of any unauthorized sharing or distribution of course materials, please report it immediately to [\[embarkxofficial@gmail.com\]](mailto:embarkxofficial@gmail.com).

5. Legal Action

We reserve the right to take legal action against individuals or entities found to be violating this usage policy.

Thank you for respecting these guidelines and helping us maintain the integrity of our course materials.

Contact Information

embarkxofficial@gmail.com

www.embarkx.com

The Ultimate Java and Spring Boot Mastery

Welcome to your one-stop-shop for mastering Java and Spring Boot! This repository offers a comprehensive learning experience with high-quality resources and community support. Dive into over 150+ hours of premium content, with everything you need to excel at Java and Spring Boot development.

Learning Roadmap

Most of the courses below are available in Udemy For Business, so if you have subscription - you can get FREE access. Here's a structured path to enhance your skills with detailed courses available:

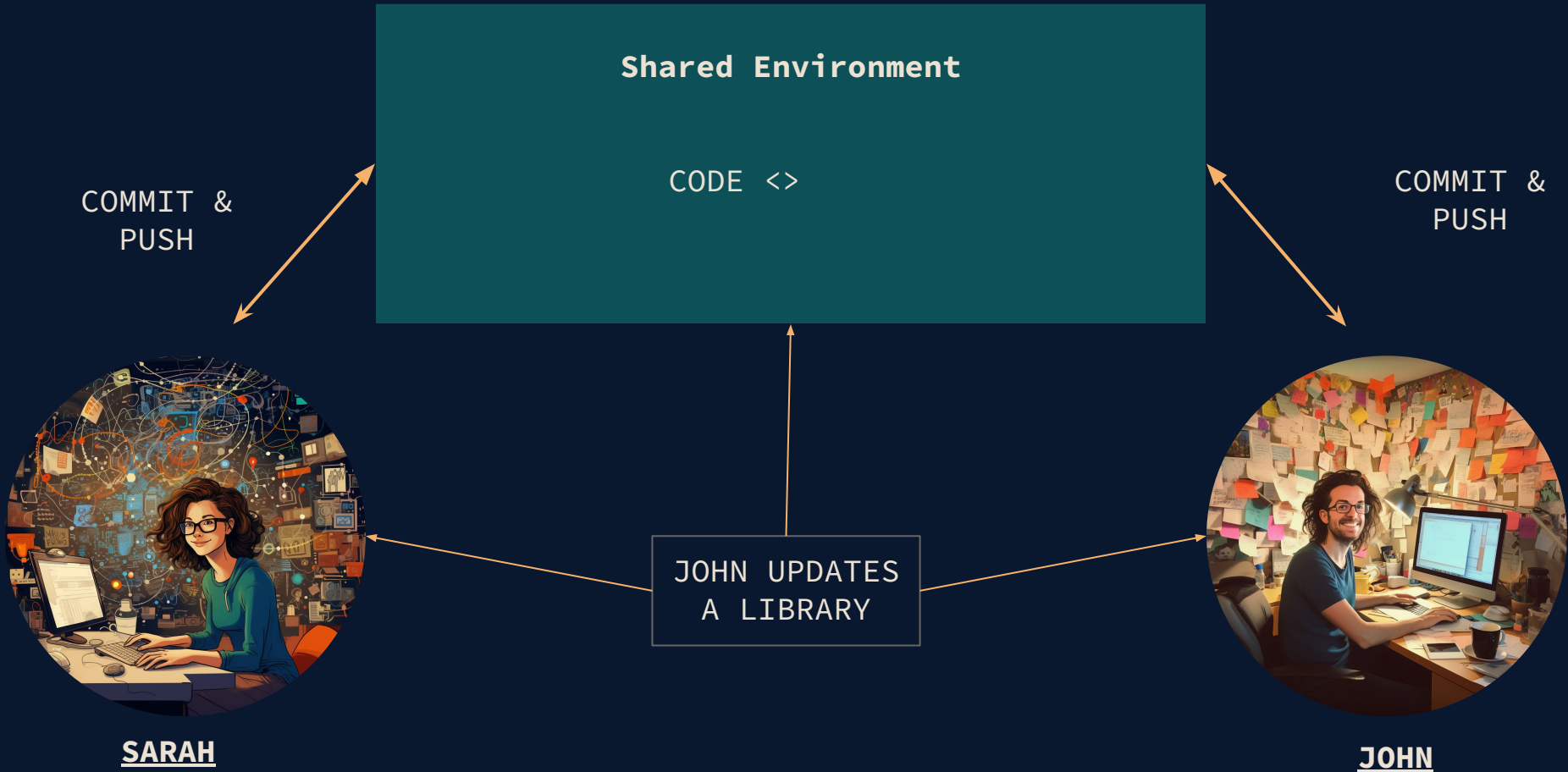
1. [Spring Boot Full Stack By Building eCommerce Application Step by Step](#) (90+ Hours of Content)
2. [Master Spring Boot Microservices](#) (70+ Hours of Content)
3. [Learn Java with 60+ Hours of Content](#) (60+ Hours of Content)
4. [Master Spring Security with React JS + OAuth2](#) (34+ Hours of Content)
5. [Master IntelliJ IDEA](#) (3+ Hours of Content)

With All Our Courses You Gain Access To

-  Notes: Detailed and downloadable notes to accompany each lesson.
-  Source Code: Full access to the source code used in the tutorials.
-  Doubt Solving: Responsive instructor and community support.
-  High-Quality HD Videos: Easy to understand, high-definition video tutorials.
-  Free Lifetime Updates: Continuous updates to course content at no extra cost.

Introduction to Docker

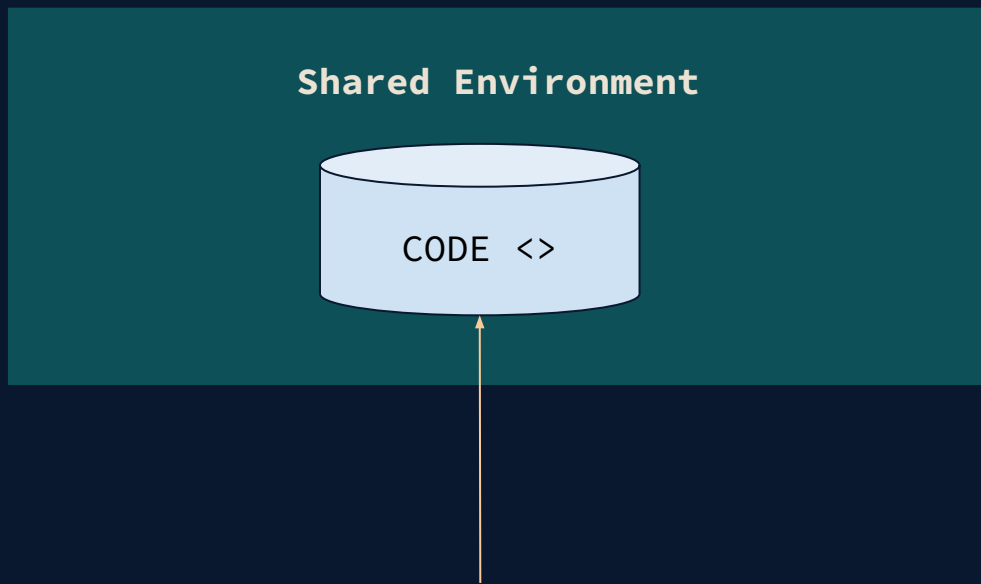
Faisal Memon (EmbarkX)



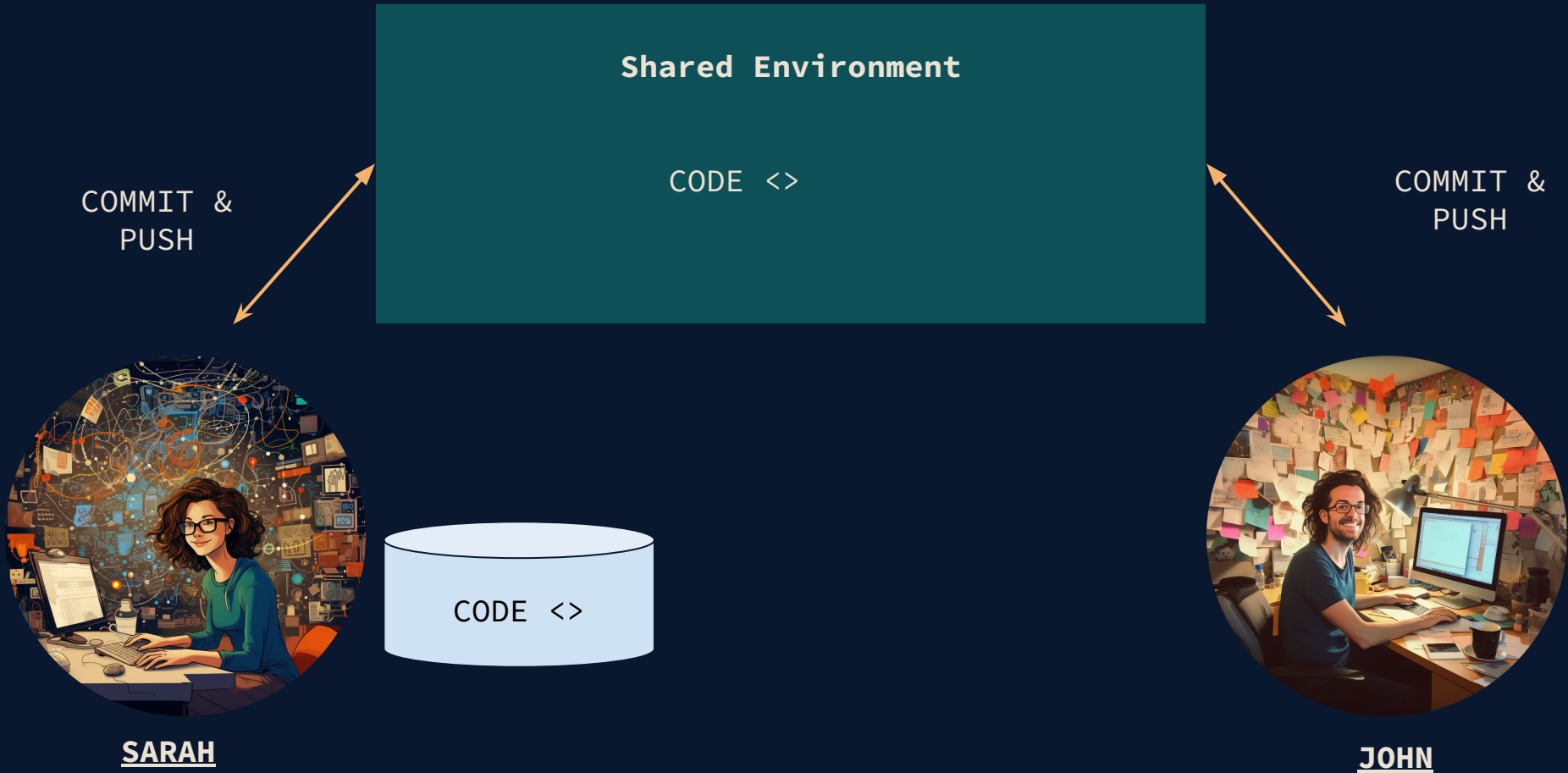
Welcome Docker

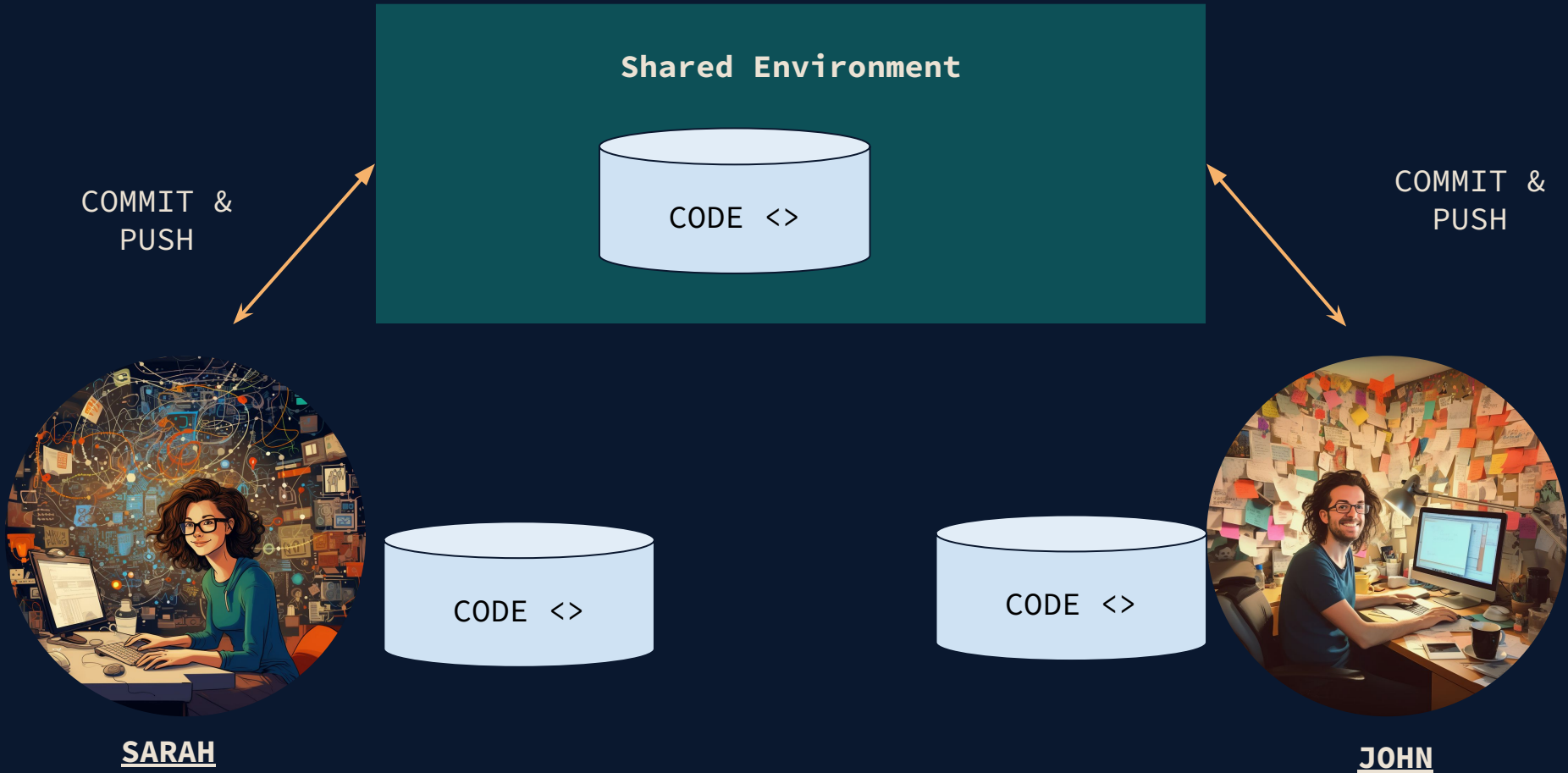
Shared Environment

CODE <>



Includes the **application code**, its **dependencies**, and the required **environment configuration**

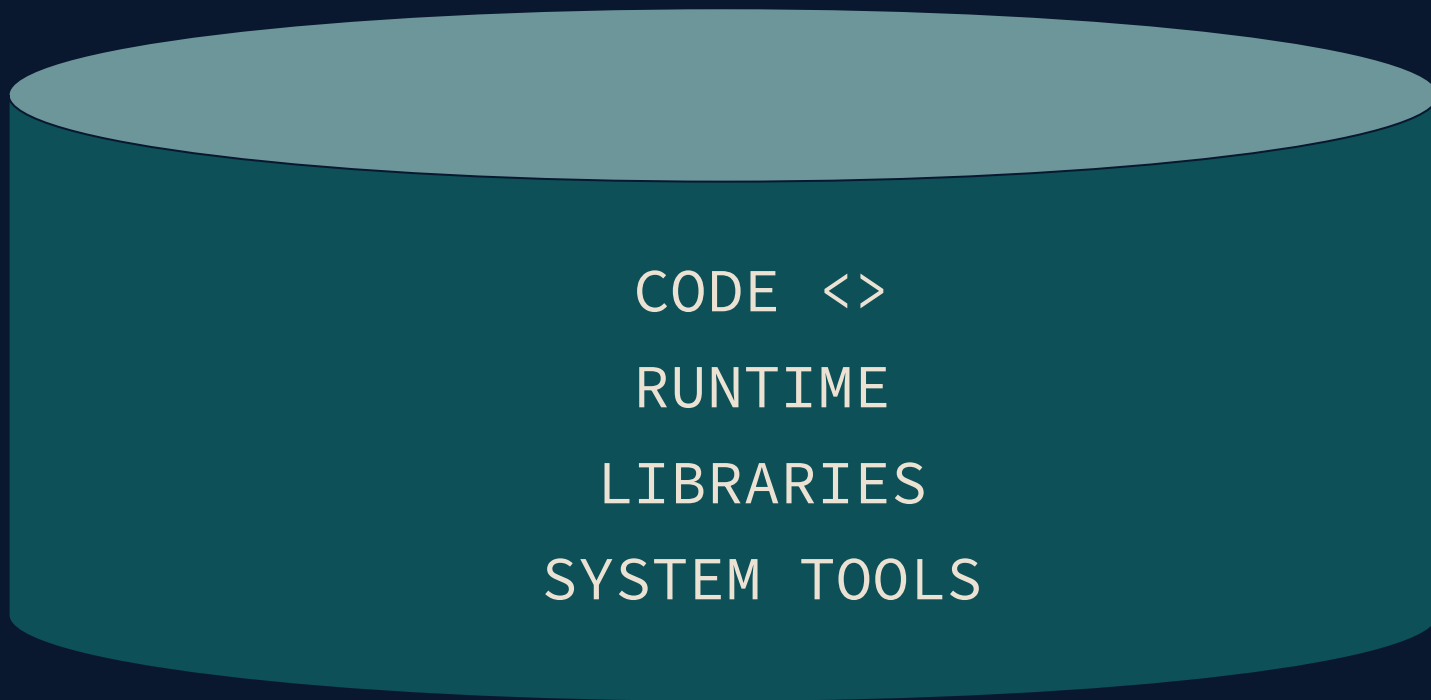




Let's Define Docker

Docker is an open-source platform that allows you to automate the deployment, scaling, and management of applications using containerization

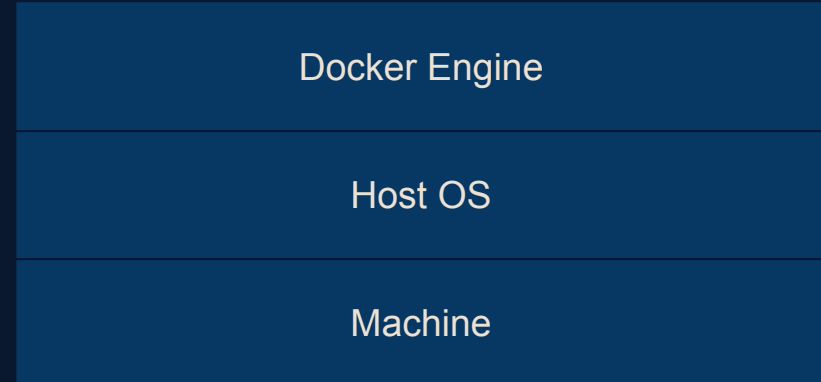
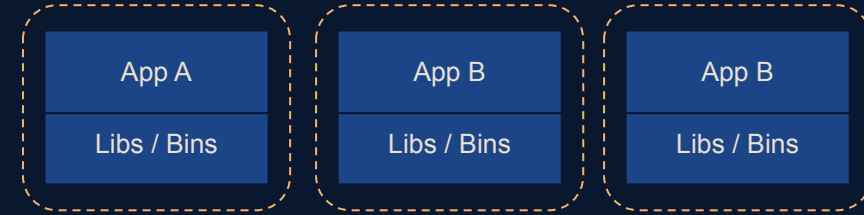
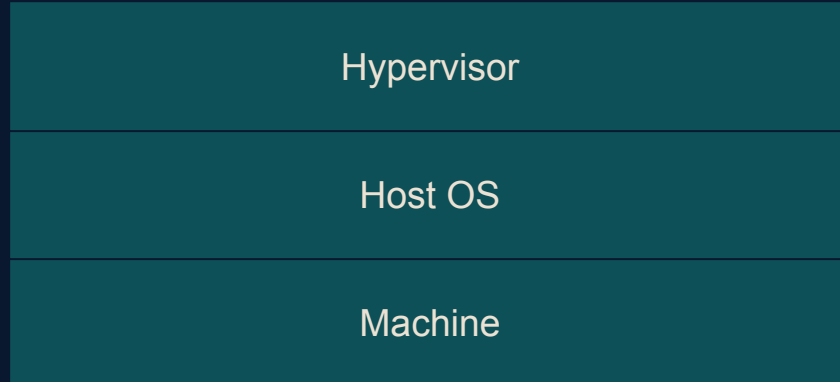
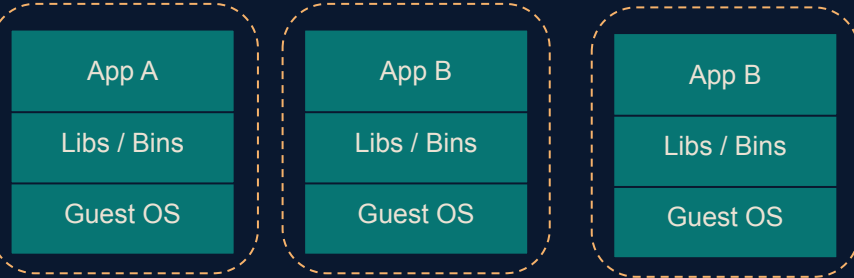
Docker Container



Virtual Machines

- *VMs act like separate computers inside your computer*
- *Each virtual machine behaves like a separate computer*
- *Virtual machines are created and managed by virtualization software*
- *They provide a flexible and scalable way to utilize hardware resources*

Docker over VM's



Parameters	Virtual Machines (VMs)	Docker Containers
<i>Size</i>	Relatively large and resource-intensive	Lightweight and resource-efficient
<i>Startup Time</i>	Longer boot time as full OS needs to start	Almost instant startup as no OS boot required
<i>Resource Utilization</i>	Utilizes more system resources (CPU, memory)	Utilizes fewer system resources
<i>Isolation</i>	Strong isolation between VMs	Isolated, but shares host OS kernel
<i>Portability</i>	Portable, but requires OS compatibility	Highly portable, independent of host OS

Parameters	Virtual Machines (VMs)	Docker Containers
<i>Scalability</i>	Scaling requires provisioning of new VMs	Easy to scale by creating more containers
<i>Ecosystem</i>	VM-specific tools and management frameworks	Docker ecosystem with extensive tooling
<i>Development Workflow</i>	Slower setup and provisioning process	Faster setup and dependency management
<i>Deployment Efficiency</i>	More overhead due to larger VM size	Efficient deployment with smaller container

Docker Concepts

Faisal Memon (EmbarkX)

- **Images**: Docker images are templates that define the container and its dependencies.
- **Containers**: Containers are runtime environments created from Docker images.
- **Dockerfile**: A Dockerfile is a file that contains instructions to build a Docker image.

Dockerfile

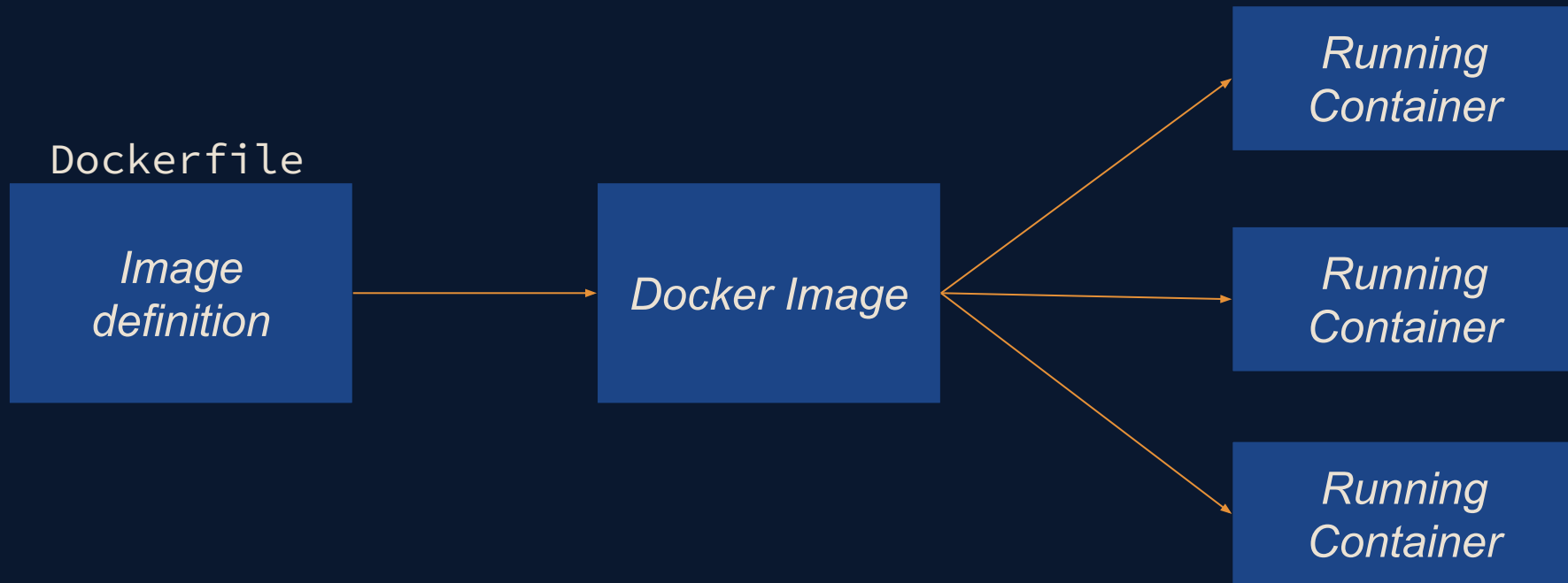
*Image
definition*



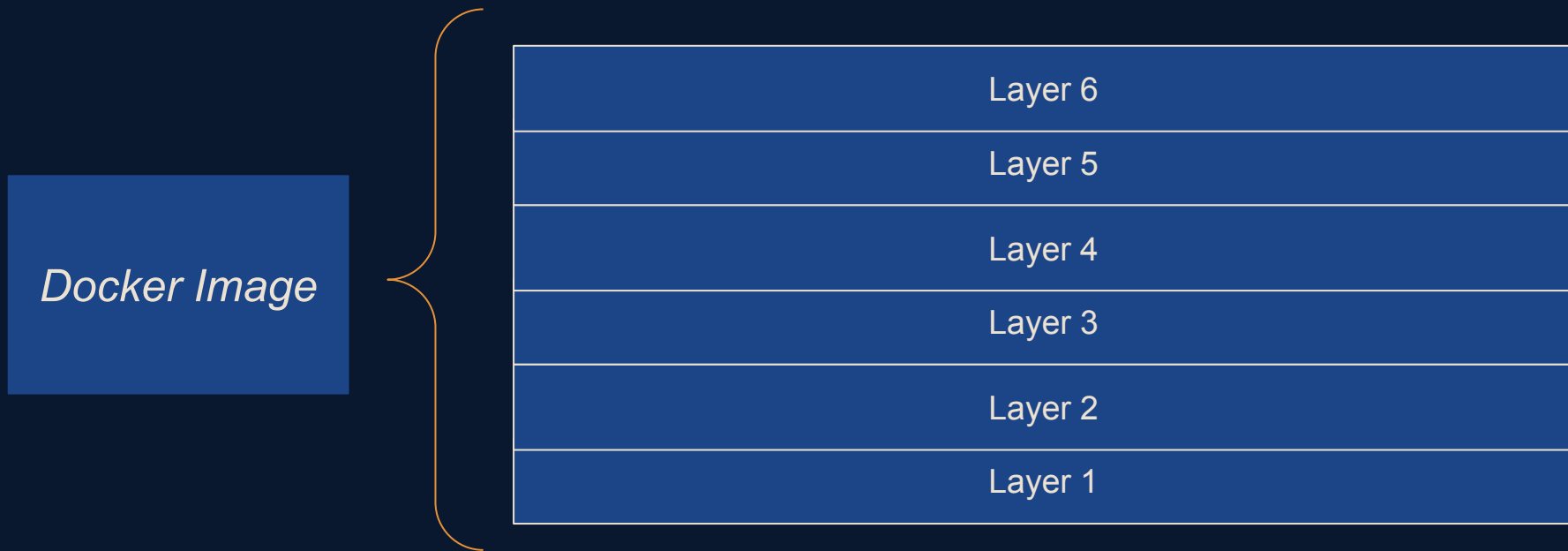
Docker Image



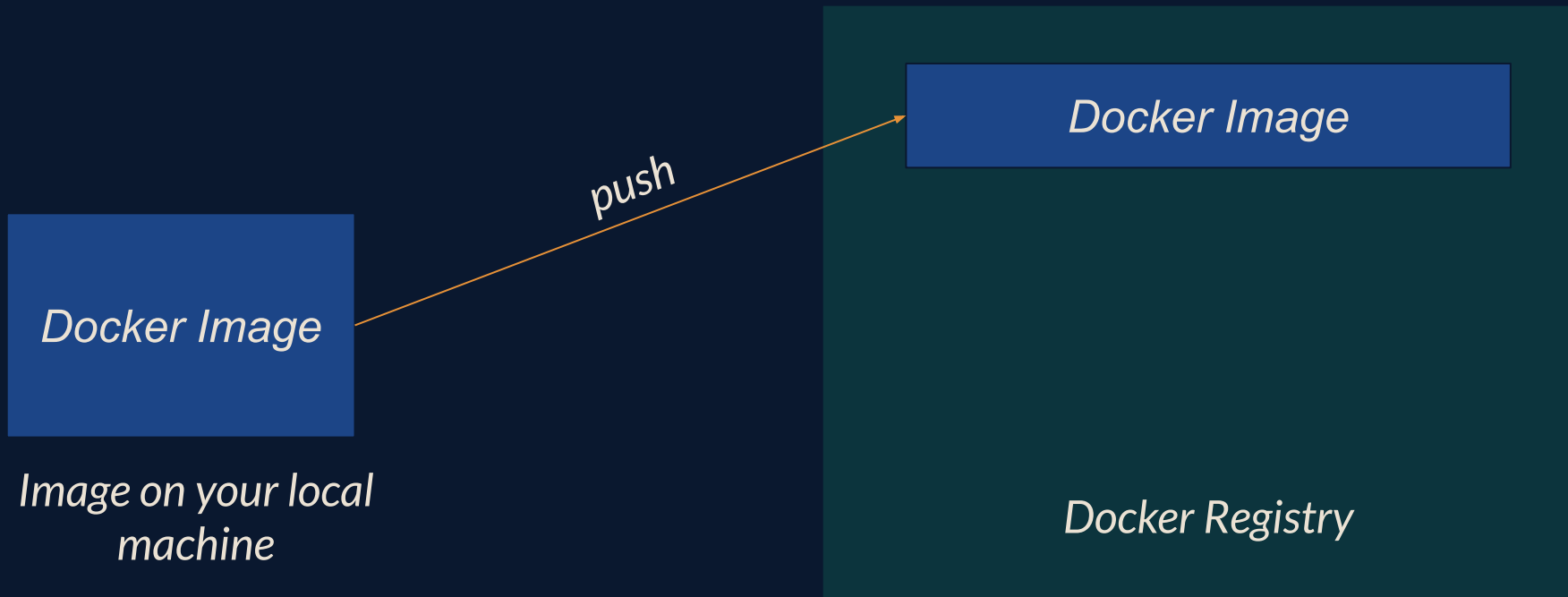
*Running
Container*



→ **Image Layer**: Every image is made up of multiple layers stacked on top of each other.



→ **Docker Registry**: A Docker Registry is just a storage service for Docker images.



→ **Docker Tag**: A tag is simply a name or label assigned to an image version.

myapp:latest

myapp:v2

myapp:prod

→ **Docker Engine**: *The Docker Engine is the runtime that runs and manages containers*

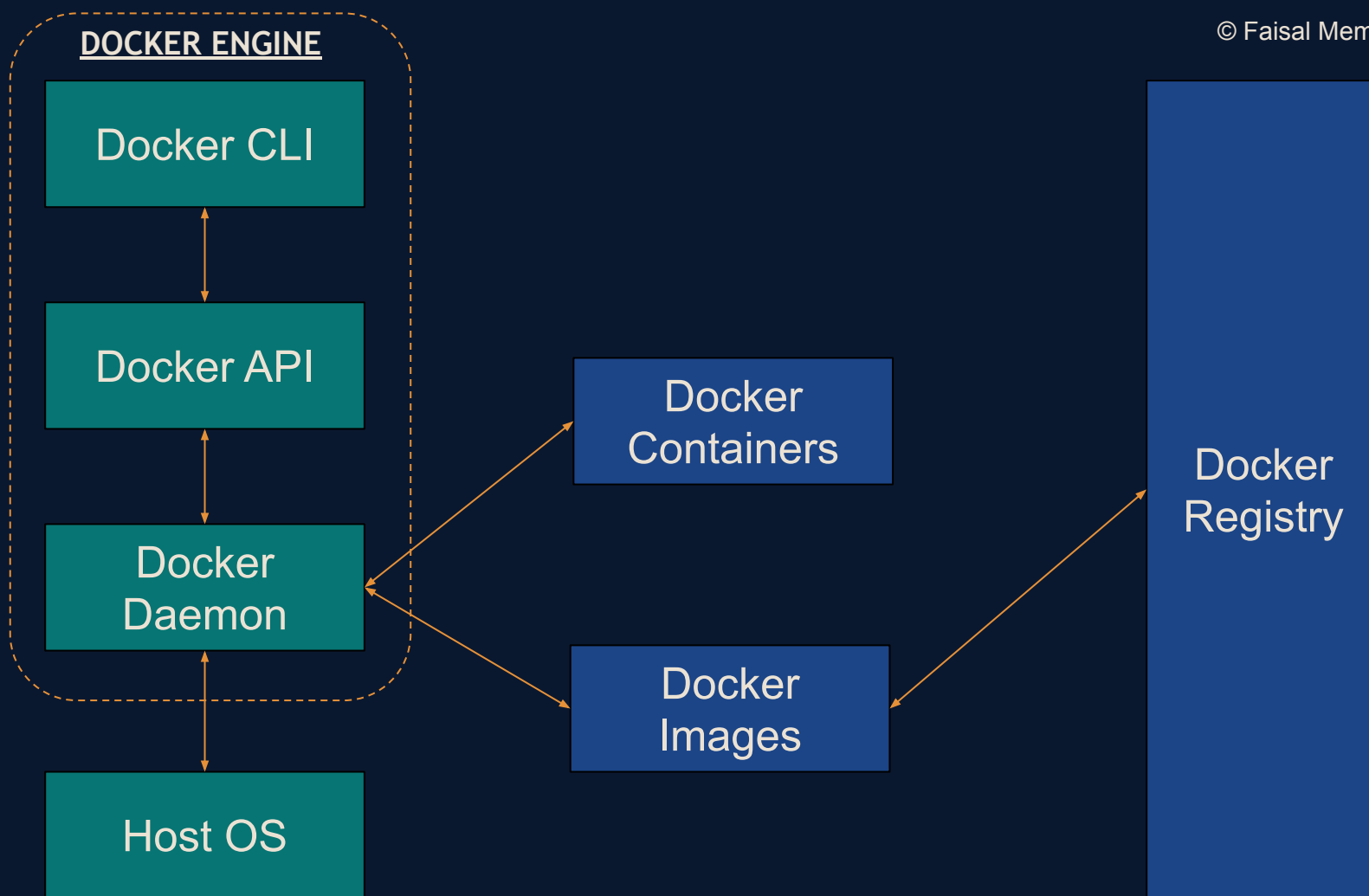
Docker Registries

Faisal Memon (EmbarkX)

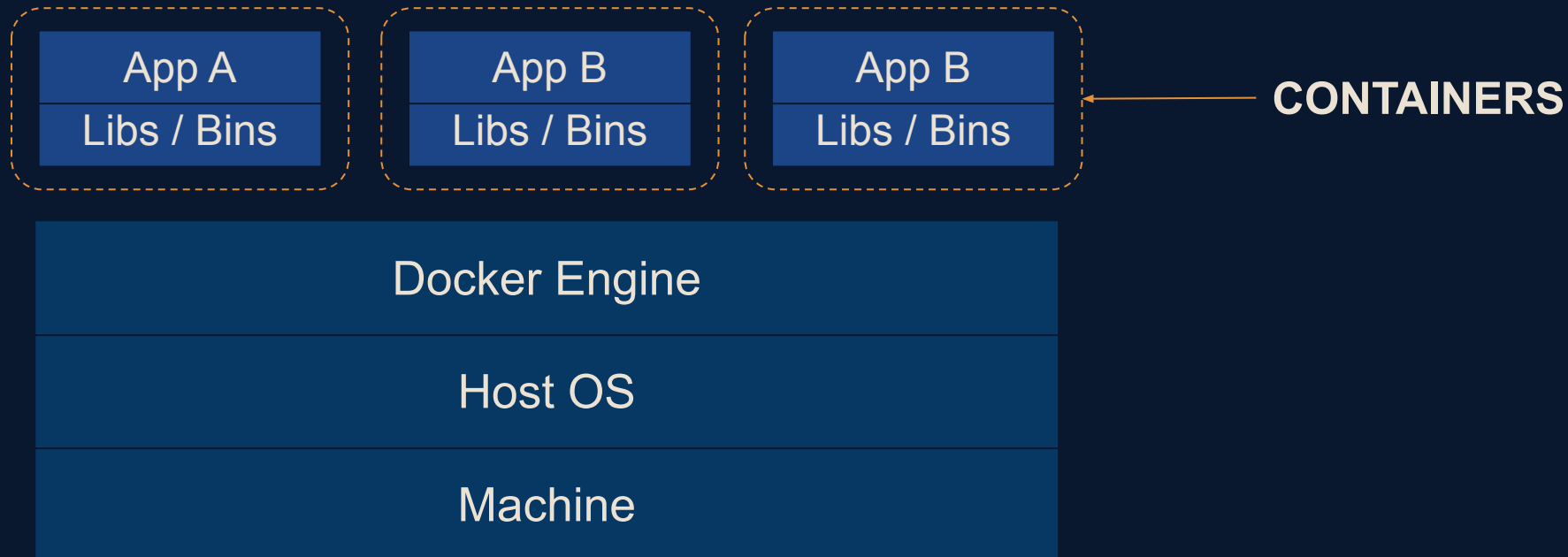
Docker Registry = stores your
container images

Docker Architecture

Faisal Memon (EmbarkX)



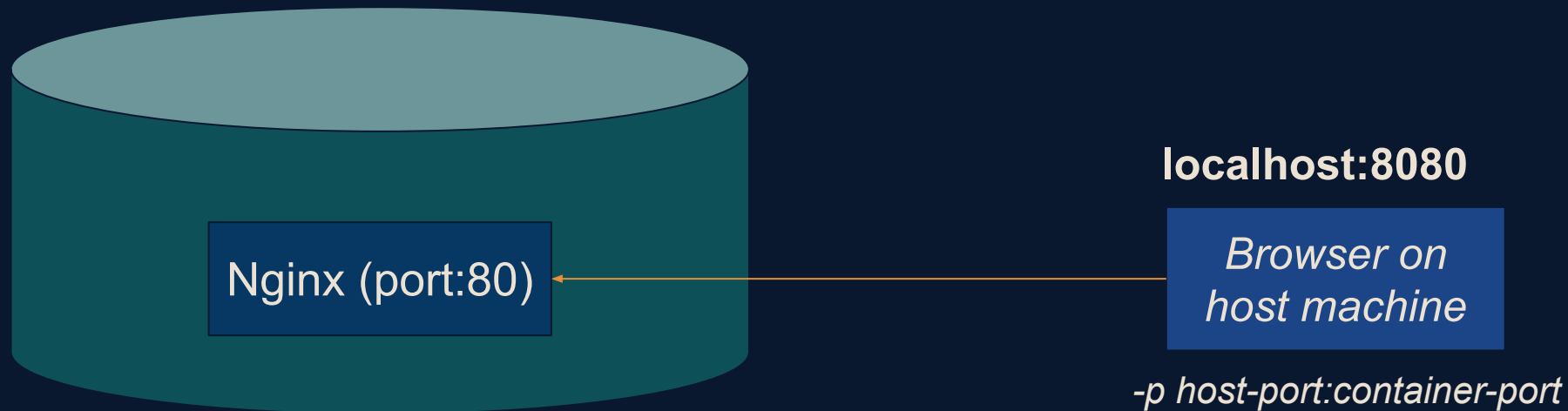
Docker



Port Mapping in Docker

Faisal Memon (EmbarkX)

Port Mapping



Container Lifecycle

Faisal Memon (EmbarkX)

Create



Start



Stop



Restart



Remove

Docker Image Tag & Versioning

Faisal Memon (EmbarkX)

Docker Tag: *A tag is simply a name or label assigned to an image version.*

myapp:latest

myapp:v2

myapp:prod

Docker Commands

Faisal Memon (EmbarkX)

Image Commands

Pull an image from a registry (Docker Hub by default)

```
docker pull <image-name>
```

e.g.: docker pull nginx:latest

List all images on your machine

```
docker images
```

Remove an image (must not be used by any container)

```
docker rmi <image-id-or-image-name>
```

Force remove image (even if in use – use carefully!)

```
docker rmi -f <image-id>
```

Build an image from Dockerfile in current folder

```
docker build -t <username/image-name:tag> .
```

e.g. docker build -t faisal/hello-node:1.0.0 .

Tag an existing image (useful before pushing)

```
docker tag <source-image> <username/repo:tag>
```

e.g. docker tag hello-node:latest faisal/hello-node:1.0.0

Container Lifecycle Commands

Create container but do NOT start it

```
docker create --name <container-name> <image>
```

Start an existing container

```
docker start <container-id-or-name>
```

Create AND start container (most used)

```
docker run <image>
```

-> docker run = create + start

Run container in interactive terminal mode

```
docker run -it <image>
```

Use when testing inside container

Run in detached mode (background)

```
docker run -d <image>
```

Container Lifecycle Commands

Run with port mapping

```
docker run -p <host-port>:<container-port> <image>
```

e.g. docker run -p 8080:80 nginx

Run with name + ports + detached

```
docker run -d --name <container-name> -p 3000:3000 <image>
```

Stop a running container (graceful)

```
docker stop <container-id-or-name>
```

Restart container

```
docker restart <container-id-or-name>
```

Remove a stopped container

```
docker rm <container-id-or-name>
```

Force remove a container (running or not)

```
docker rm -f <container-id-or-name>
```

Inspecting & Debugging Containers

View all running containers

```
docker ps
```

View ALL containers (running + stopped)

```
docker ps -a
```

Check container logs (useful for debugging)

```
docker logs <container-id-or-name>
```

Follow logs in real-time

```
docker logs -f <container-id-or-name>
```

Jump inside a running container

```
docker exec -it <container-name> bash
```

If container doesn't have bash, try:

```
docker exec -it <container-name> sh
```

Inspect container details (ports, env, mounts, etc.)

```
docker inspect <container-id-or-name>
```


Cleanup & Removing Everything

Remove ALL stopped containers

```
docker container prune
```

Remove ALL images not used by containers

```
docker image prune
```

Remove all unused images, containers, networks, and build cache

```
docker system prune
```

Remove EVERYTHING (images + containers + networks + cache)

```
docker system prune -a
```

-a removes unused images too

Remove all containers

```
docker rm $(docker ps -aq)
```

Remove all images

```
docker rmi $(docker images -q)
```

Remove all networks (not default ones)

```
docker network prune
```

System Information Commands

Docker version

`docker --version`

`docker version`

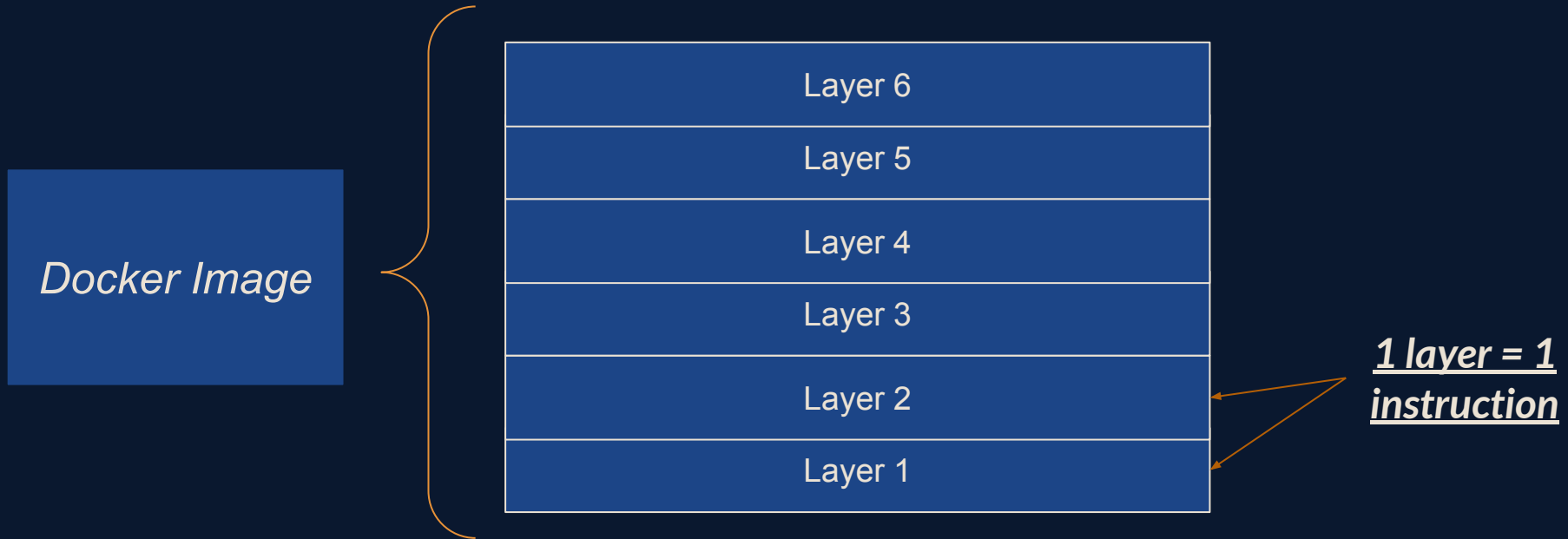
Docker system-level info

`docker info`

Understanding Dockerfile

Faisal Memon (EmbarkX)

Image Layer: *Every image is made up of multiple layers stacked on top of each other.*



Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```


Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Dockerfile

```
FROM eclipse-temurin:21-jdk
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Essential Dockerfile Instructions

FROM – The Base Image

Sets the starting point for your container.

```
FROM node:20
FROM python:3.10
FROM openjdk:17-jdk
```

COPY – Bring Your App Code Into the Image

COPY moves your application files from your computer to the image. Without COPY, your final image would be empty.

```
COPY . .
COPY package.json .
COPY app.py .
COPY target/myapp.jar app.jar
```

Essential Dockerfile Instructions

RUN — Execute Commands During Image Build

RUN executes commands while building the image.

```
RUN npm install  
RUN pip install -r requirements.txt  
RUN ./mvnw clean package
```

CMD — What To Run When The Container Starts

CMD defines the default command that runs when your container starts.

```
CMD ["node", "app.js"]  
CMD ["python", "app.py"]  
CMD ["java", "-jar", "app.jar"]
```

```
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Dockerfile for Spring Boot app

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 3000
CMD ["python", "main.py"]
```

Dockerfile for Python app

```
FROM node:18-slim
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Dockerfile for Node js app

.dockerignore

Faisal Memon (EmbarkX)

docker build .

→ *Docker sends your entire folder (called build context) to the Docker engine.*

→ *.dockerignore works exactly like .gitignore, but for Docker.*

Why .dockerignore is critical

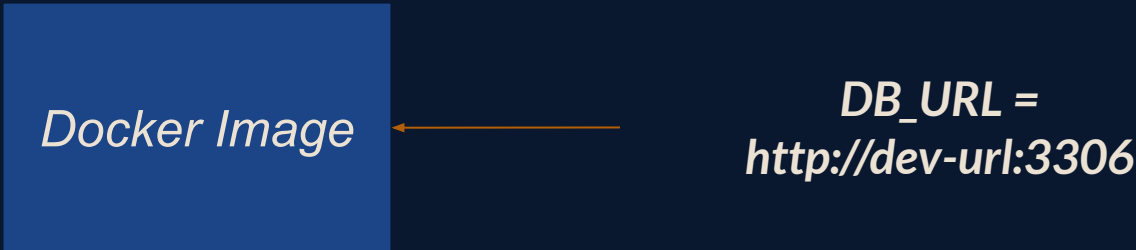
- *Speeds up docker build*
- *Makes images smaller*
- *Prevents security issues*
- *Avoids broken builds*

Using Environment Variables in Containers

Faisal Memon (EmbarkX)

Environment Variables: *named values that store dynamic data outside of your application code*

Docker Image



A diagram illustrating the concept of environment variables in a Docker context. On the left, a blue rectangular box contains the text "Docker Image". To its right, the text "DB_URL = http://dev-url:3306" is displayed. A thin orange arrow points from the text on the right towards the blue box, indicating that the environment variable is being passed to or used by the Docker image.

DB_URL =
http://dev-url:3306

Docker Compose

Faisal Memon (EmbarkX)

Docker Compose is a tool
for defining and running
multi-container applications
using a **YAML** file

How many containers exist in
real-world production
grade apps?

Typically

→ *Frontend + API Gateway*

→ *Cache + Message Broker*

→ *Backend services (User, Order, Payment)*

→ *Database (PostgreSQL / MySQL)*

→ *Auth Server, Observability, Logging (ELK Stack)*

Life WITHOUT Docker Compose

```
docker run postgres
```

```
docker run redis
```

```
docker run kafka
```

```
docker run user-service
```

```
docker run order-service
```

```
docker run payment-service
```

```
docker run frontend
```

```
docker run nginx
```

→ *Correct startup order*

→ *Correct networking*

→ *Correct environment variables*

→ *Correct ports*

Docker Compose Says

*Here is my entire application stack.
Run it together. Stop it together.*

Compose = Run multiple containers as ONE application

docker compose up

Single Container

VS

Multi-Container

→ Just have one service running

→ Learning Docker basics, testing one service

→ It's a simple app with 1 service

→ Docker Compose not mandatory

→ Multiple services Frontend, Backend, Database

→ This is how production grade apps function

→ Each service scales independently, each can be written in different languages, matches real production systems

→ Docker Compose mandatory to manage this efficiently

docker-compose.yml

```
services:
  app:
    image: my-backend-app
    ports:
      - "8080:8080"

  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
```

A compose file describes:

- *What containers you need*
- *How they talk to each other*
- *What ports they expose*
- *What environment variables they need*

Docker Compose Lifecycle

docker compose up

Starts everything. Creates network, starts containers in correct order, shows logs in terminal

docker compose down

Stops and cleans up. Stops containers, removes containers, removes network

docker compose logs

View logs of all services. Debugging, teaching logs concept, showing errors live.

Docker Compose Lifecycle

`docker compose ps`

See running services. Which containers are running, port mappings, status.

Key Takeaways

- *Docker = runs one container*
- *Docker Compose = runs multiple containers together*
- *Compose uses a YAML file*
 - up → start app*
 - down → stop app*
 - logs → debug*
 - ps → inspect status*

How Many Containers Can a Production-Grade App Have?

Anywhere from 5 to 100+ containers

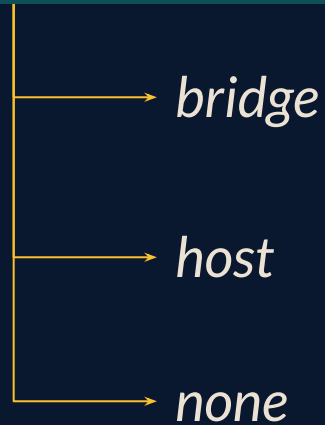
*Netflix-scale systems → **1000s** of containers (managed via Kubernetes)*

App Type	Containers
Small startup app	5-10
Medium SaaS	15-30
Large monolith + infra	20-40
Microservices system	50-200+

Docker Networks

Faisal Memon (EmbarkX)

Networks Created By Default



bridge (the default network)

- Any container you create without specifying a network goes here automatically
- Example: When you run “docker run nginx”, it connects to this bridge network
- Containers can talk to each other using IP addresses, but not by container names

host

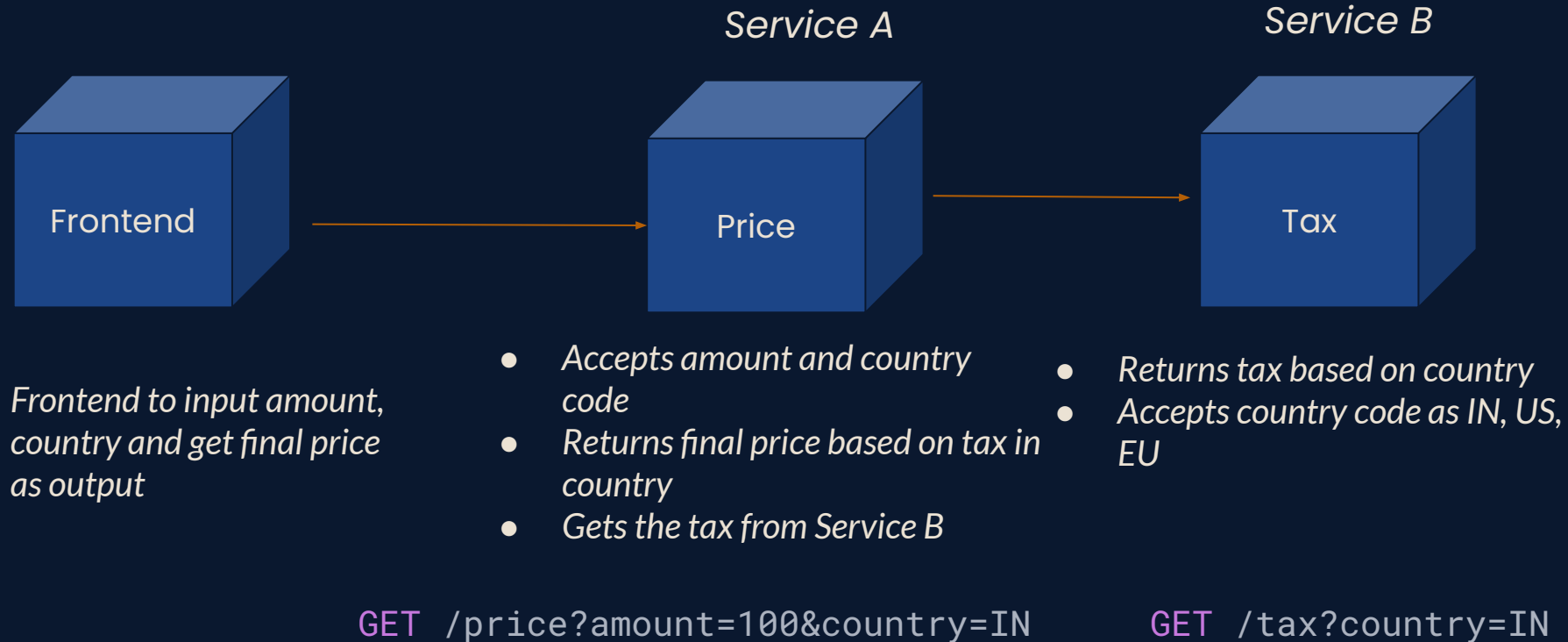
- *The container shares your computer's network directly*
- *When you need maximum performance or want the container to act like it's running directly on your machine*
- *Container uses your computer's IP address*
- *Ports are directly exposed on your machine*
- *Use it when you need maximum network performance (no translation overhead)*

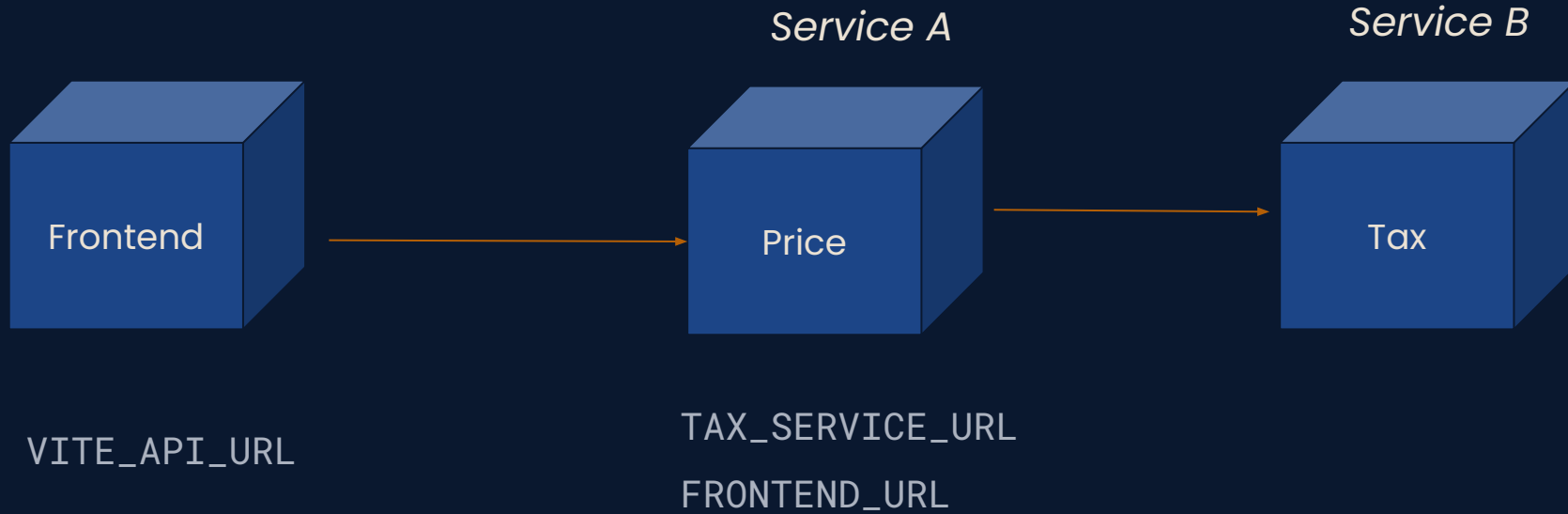
none

- *A container with no network connection at all.*
- *Security testing, Batch jobs, Work that need maximum security or isolation*
- *No IP, No outbound / inbound traffic*

Understanding Our Microservices

Faisal Memon (EmbarkX)

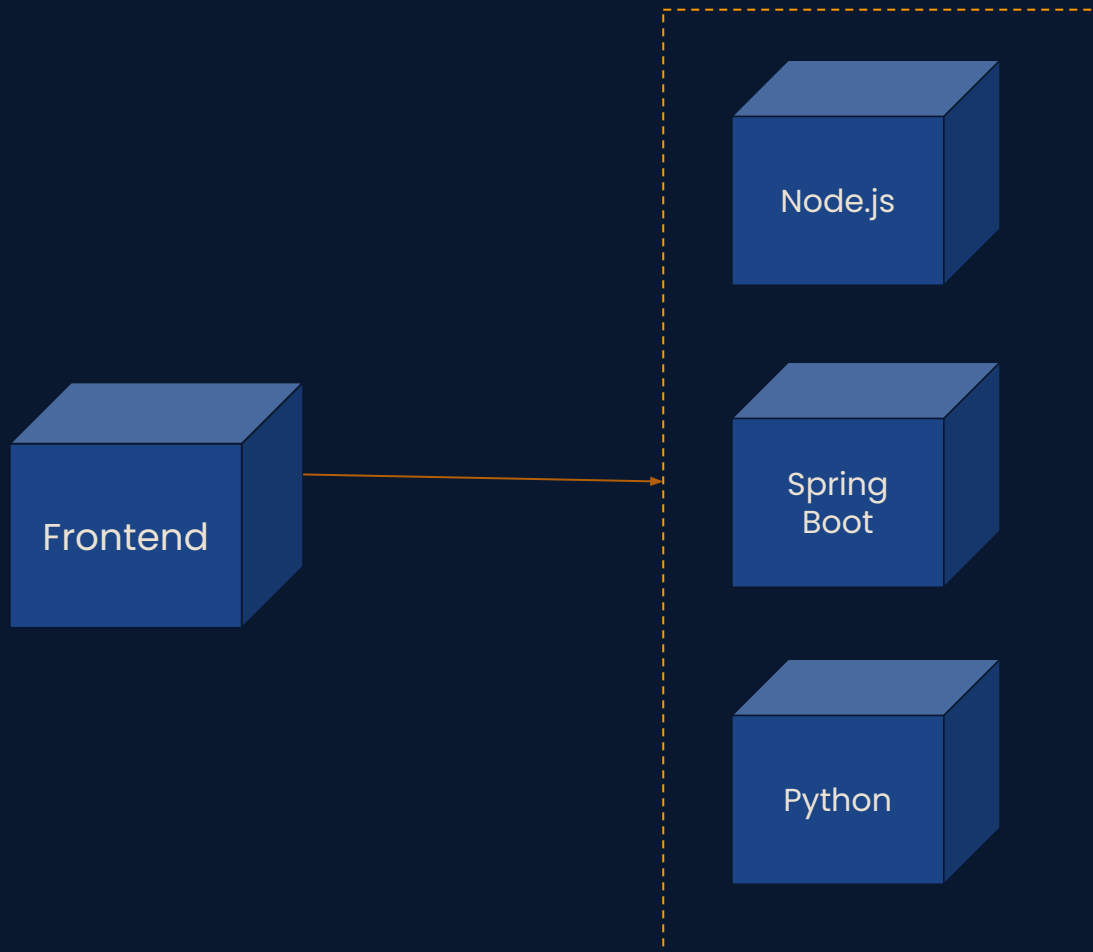




What Is Kubernetes & Why It Exists

Faisal Memon (EmbarkX)

Docker solved “***it works on my machine***” by packaging apps + dependencies into containers.



WHAT HAPPENS IF?

- A container crashes at 2 AM?
- Traffic suddenly jumps 10×?
- You need 5 instances of service-A?
- One container runs on another VM?
- You deploy a new version without downtime?

What is Orchestration?

Orchestration means automatically managing many containers across machines — including running, scaling, networking, and healing them.

Docker → runs one container

Orchestration → manages many containers together

Chefs



Dishes



Kitchen Manager





Orchestration is like a manager coordinating workers so work continues smoothly even when things go wrong.

The Container Orchestration Problem

Problem	Example
Container crashes	Who restarts it?
Scaling	How to run 10 copies?
Load balancing	Which container gets traffic?
Service discovery	How does service-A find service-B?
Zero downtime deploy	How to update without outage?
Multi-machine setup	Docker works per machine
Health checks	How to detect broken containers?

Kubernetes

Kubernetes is

A container orchestration platform that manages containers for you at scale.

→ *Kubernetes automates deployment, scaling, healing, networking, and management of containerized applications.*

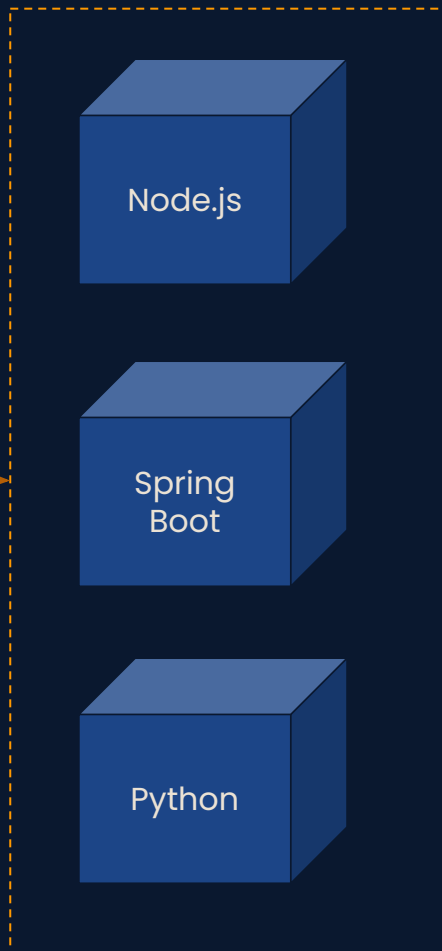
Docker vs Kubernetes

Docker	Kubernetes
Runs containers	Manages containers
Single host focus	Multi-node clusters
Manual scaling	Auto scaling
Manual restart	Self-healing
Manual networking	Built-in service discovery

→ *Docker is the engine*

→ *Kubernetes is the autopilot*

Without Kubernetes

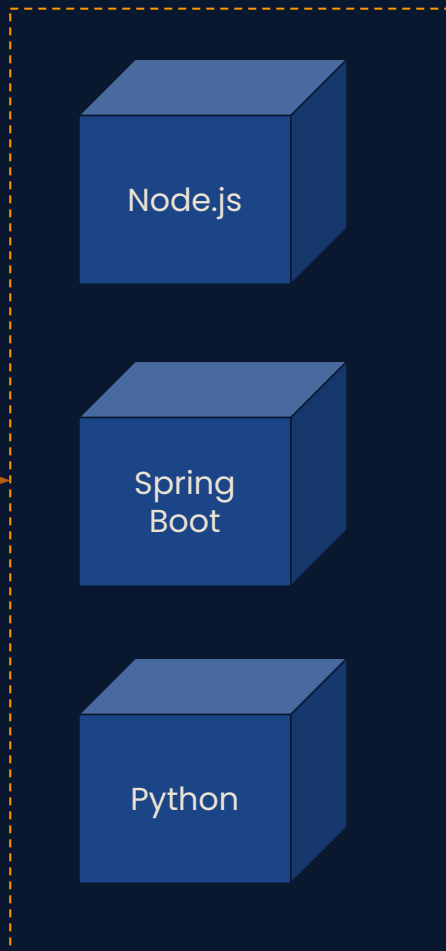


WHAT HAPPENS IF?

- Nodejs service crashes
- App breaks

- Ops team wakes up
- SSH
- restart container 🥵

With Kubernetes



KUBERNETES WILL

- *Detects crash*
- *Restarts container*
- *Maintains desired count (e.g. 3 replicas)*
- *Routes traffic only to healthy pods*
- *Scales up/down based on load*

👉 *No human intervention needed*

Key Problems Kubernetes Solves

Self-Healing: Container crashes?

- Kubernetes restarts it automatically
- If node dies → pod moves to another node

Scaling: Traffic spike?

- replicas: 3 → 10
- Manual or auto-scaling
- Based on CPU / memory / metrics

Service Discovery & Networking

- Instead of: `http://192.168.1.12:3000` , You get: `http://service-a`
- Built-in DNS
- Stable service names

Key Problems Kubernetes Solves

Zero-Downtime Deployments: *Deploy v2 without breaking users*

- *Rolling updates*
- *Blue-Green / Canary deployments*

Infrastructure Abstraction: *Your app doesn't care*

- *Which VM? Which cloud? Which IP?*
- *Kubernetes handles it.*

Why Every Company Uses Kubernetes

→ *Because modern systems are:*

Microservices-based, Cloud-native, Distributed, Always-on

→ *Kubernetes gives:*

Reliability, Scalability, Portability (AWS, GCP, Azure, on-prem)

→ *That's why:*

Startups use it, Enterprises standardize on it, DevOps engineers must know it

Docker packages applications.
Kubernetes runs and manages
them reliably at scale.

History of Kubernetes

Faisal Memon (EmbarkX)

History of Kubernetes

The real problem (inside Google)

In the early 2000s, Google was running millions of applications (Search, Gmail, YouTube). Manually managing servers was impossible.

Google built Borg (internal system)

Google built an internal system called Borg. Borg worked so well that Google ran everything on it.

Containers became popular (Docker era 2013-2014)

Docker made containers easy for everyone but Docker could run containers. It could NOT manage them at scale.

History of Kubernetes

Kubernetes is born (2014)

Kubernetes = Borg ideas + containers + open source

Why Kubernetes exploded in popularity

Because it solved real production problems, worked on any cloud (AWS, GCP, Azure), was open-source (no vendor lock-in)

Kubernetes was Donated

In 2015, Kubernetes was donated to Cloud Native Computing Foundation (CNCF). This was done to ensure it was vendor-neutral and not controlled by a single entity.

History of Kubernetes

Kubernetes Today

- *Kubernetes is the industry standard*
- *Almost every company uses it directly or indirectly*

What does K8s mean?

→ Kubernetes is long to say 😄 So engineers shorten it to:

$K + 8 \text{ letters} + s = \underline{K8s}$

Kubernetes = K8s

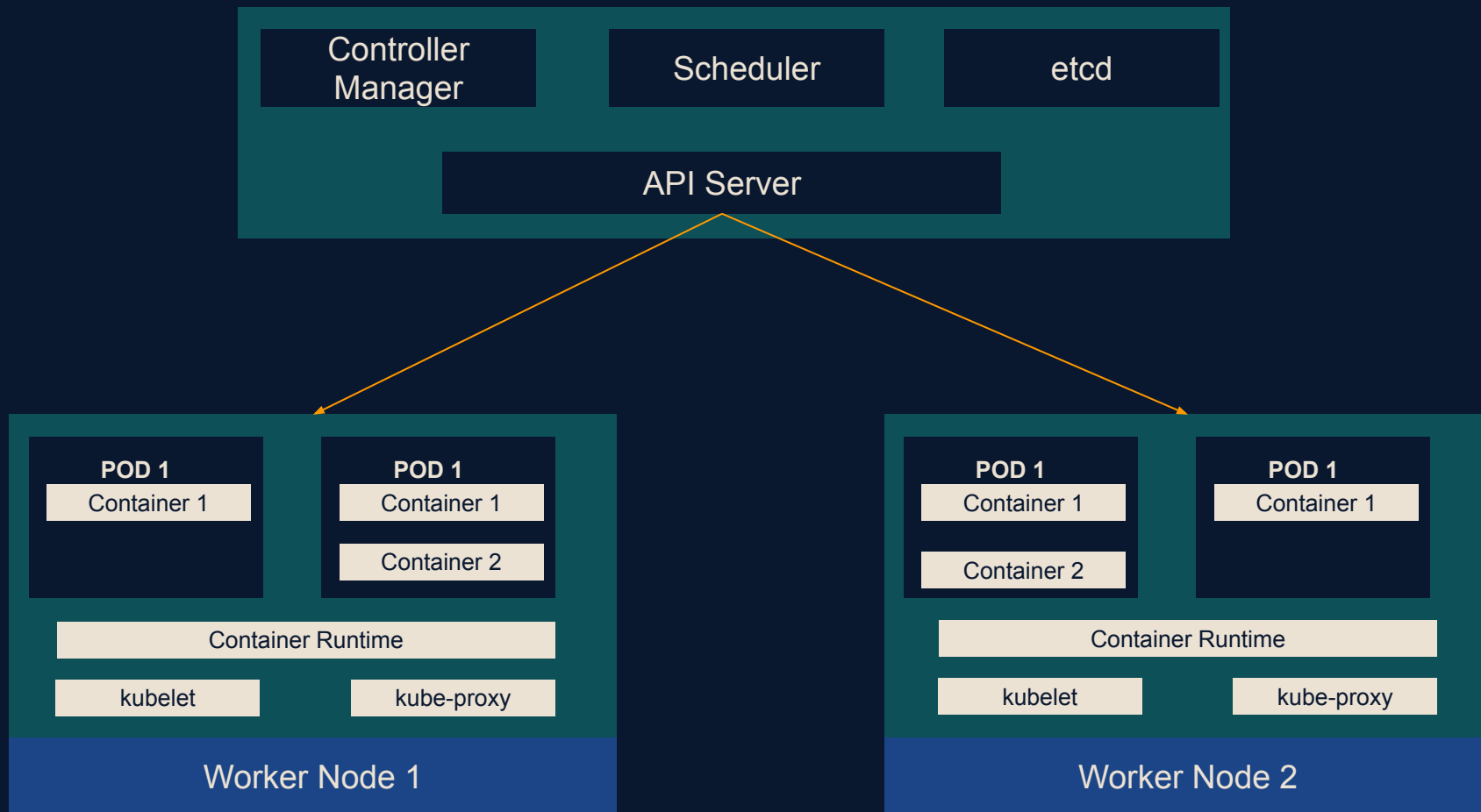
Kubernetes started at Google to solve **large-scale container management** problems and **became the global standard** for running applications reliably in production.

Kubernetes Architecture

Faisal Memon (EmbarkX)

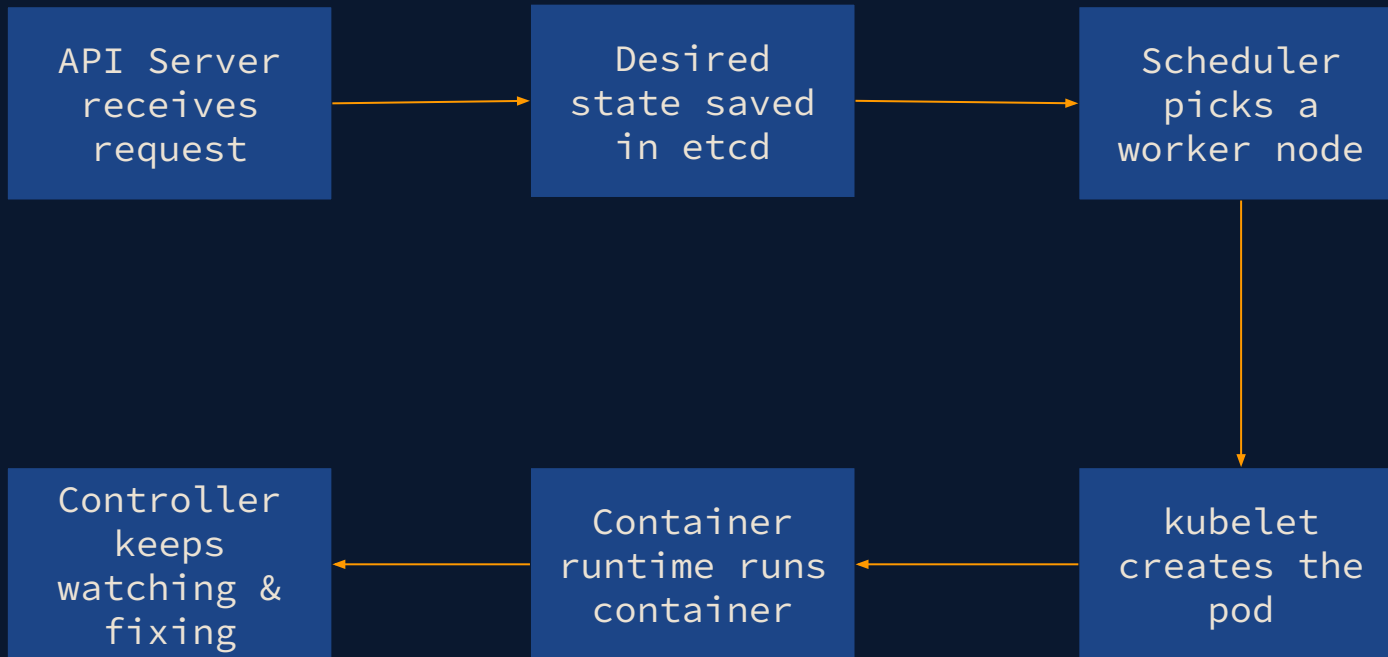
Kubernetes Cluster = Control Plane +
Worker Nodes

CONTROL PANE



Control Plane → decides & manages
Worker Nodes → run your applications

kubectl apply -f app.yaml



Setting Up k8s Locally

Faisal Memon (EmbarkX)

Setting up k8s locally



Pods and Deployments

Faisal Memon (EmbarkX)

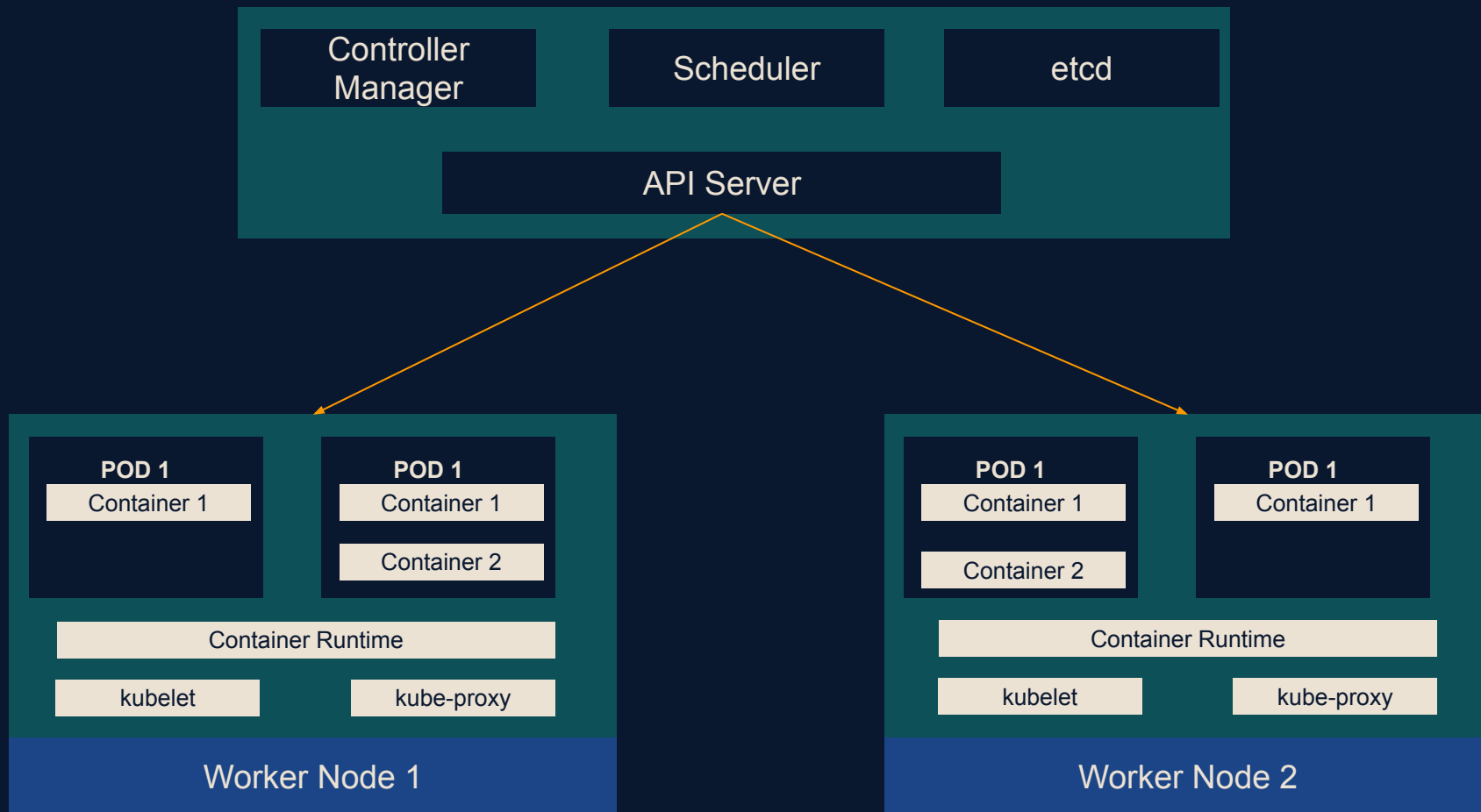
Pod

- *The actual running app (container wrapped in Kubernetes layer)*
- *Pod is where your app actually lives and runs.*
- *It's the smallest deployable unit in Kubernetes.*

Deployment

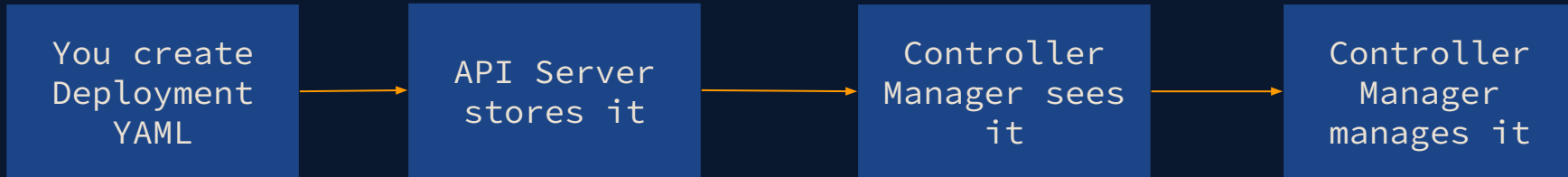
- *It runs your app continuously.*
- *If the pod crashes, Kubernetes restarts it automatically.*
- *If you want more copies (scaling), Deployment handles it.*
- *If you want zero downtime updates, Deployment does rolling updates.*

CONTROL PANE



Deployment vs Controller Manager

- *Deployment is your instruction / desired state*
- *Controller Manager (Deployment Controller) is the worker that makes it happen*



Services in Kubernetes

Faisal Memon (EmbarkX)

Service in k8s

Without a Service:

- *Your pod is NOT accessible*
- *Your app is invisible outside the cluster*
- *Every time pod restarts, new IP address is assigned & old connection breaks*

What Service does:

- *Gives your app a permanent IP*
- *Lets other apps inside the cluster talk to it*
- *Lets you expose it to the browser (NodePort)*

👉 Service = “Give my app a stable address so browsers and other apps can reach it.”

Service Types



Service Types

ClusterIP (Default)

Exposes app inside the cluster only, won't be accessible from browser. Useful when - backend talking to backend & Microservices communication

NodePort

Exposes app on a node's IP + port. Makes app accessible from browser. Port range is of 30000–32767. Used generally for local testing, demos & learning Kubernetes

LoadBalancer

Creates a cloud load balancer and gives public IP / DNS. Works best on Cloud Services like AWS, GCP, Azure. Used majorly for production apps with real users

Service Types

ExternalName

Maps Service to an external DNS. Example: db.example.com

Used for external db or third party API.

Summary

Type	Accessible From	When to Use
ClusterIP	Inside cluster	Microservices
NodePort	Browser (IP:Port)	Local / learning
LoadBalancer	Public Internet	Production
ExternalName	External DNS	Outside services

Summary

Deployment → creates & manages Pods

Pod → runs your app (nginx)

Service → gives a stable connection to the Pod

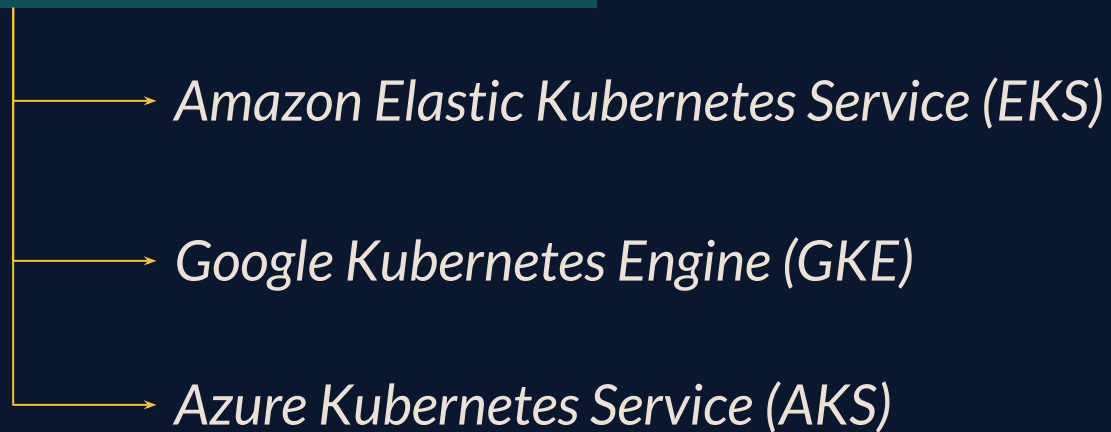
NodePort → lets you access the Service from your laptop



Kubernetes on AWS Cloud with Amazon EKS

Faisal Memon (EmbarkX)

Kubernetes in Cloud

- 
- ```
graph LR; A[Kubernetes in Cloud] --> B[Amazon Elastic Kubernetes Service (EKS)]; A --> C[Google Kubernetes Engine (GKE)]; A --> D[Azure Kubernetes Service (AKS)];
```
- *Amazon Elastic Kubernetes Service (EKS)*
  - *Google Kubernetes Engine (GKE)*
  - *Azure Kubernetes Service (AKS)*



# Kubernetes on AWS

*Means running Kubernetes using AWS infrastructure instead of your own servers*

# Kubernetes on AWS

- *AWS provides a managed Kubernetes service called Amazon Elastic Kubernetes Service (EKS)*
- *AWS runs the Kubernetes control plane for you*
- *AWS handles availability, upgrades, security of K8s itself*

# *Why not run Kubernetes yourself?*

*Running Kubernetes Yourself = High operational burden*

# Kubernetes on AWS

***Your App (Containers)***

*You talk to Kubernetes*



***Kubernetes (EKS)***

*Kubernetes talks to AWS*



***AWS Infrastructure***

*AWS runs everything using its infrastructure (EC2, Networking, Load Balancers)*

# Key AWS services involved

## *Amazon EC2*

→ *Virtual machines where your containers actually run*

## *Amazon VPC*

→ *Private network for your cluster*

## *AWS IAM*

→ *Who can access what (security)*

## *Elastic Load Balancing*

→ *Exposes your app to the internet*

## AWS Manages

&

## You Manage

- Kubernetes API Server
- etcd (cluster data)
- High availability (multi-AZ)
- Control plane security & patches
- and more...

- Your application containers
- Kubernetes configs (Deployments, Services, etc.)

## Things to know

- *Kubernetes is not AWS-specific*
- *EKS is AWS's managed way to run Kubernetes*
- *Containers still run on VMs*
- *Kubernetes orchestrates, AWS provides infra*

# Clusters, Nodes with Local vs Cloud K8s Setup

---

Faisal Memon (EmbarkX)



# Node & Clusters

## Node

- Node = one Virtual Machine
- Node has CPU, Memory, Network, Runtime (container runtime)
- Pods never run directly on cluster, they always run on a node
- If node dies, pods on that node also die

## Cluster

- Cluster = control plane + one or more nodes
- Cluster  $\neq$  machine
- Cluster can span 1 node or 1000 nodes
- You deploy apps to the cluster, not to nodes directly

*Cluster*

```
graph LR; Cluster[Cluster] --> Node1[Node]; Cluster --> Node2[Node]; Cluster --> Node3[Node]; Node1 --> Pod1_1[Pods]; Node1 --> Pod1_2[Pods]; Node1 --> Pod1_3[Pods]; Node2 --> Pod2_1[Pods]; Node2 --> Pod2_2[Pods]; Node2 --> Pod2_3[Pods]; Node3 --> Pod3_1[Pods]; Node3 --> Pod3_2[Pods]; Node3 --> Pod3_3[Pods];
```

The diagram illustrates the Kubernetes architecture. At the top left is a pink box labeled 'Cluster'. A vertical line descends from the bottom of the 'Cluster' box, with three horizontal arrows pointing to three separate light yellow boxes, each labeled 'Node'. From the bottom of each 'Node' box, a horizontal line extends to the right, with three vertical arrows pointing to three separate yellow boxes, each labeled 'Pods'. This structure shows that a single cluster contains multiple nodes, and each node contains multiple pods.

*Node*

*Pods*

*Pods*

*Pods*

*Node*

*Pods*

*Pods*

*Pods*

*Node*

*Pods*

*Pods*

*Pods*

## Local vs Cloud Setup

*1 cluster → 1 node → many pods*

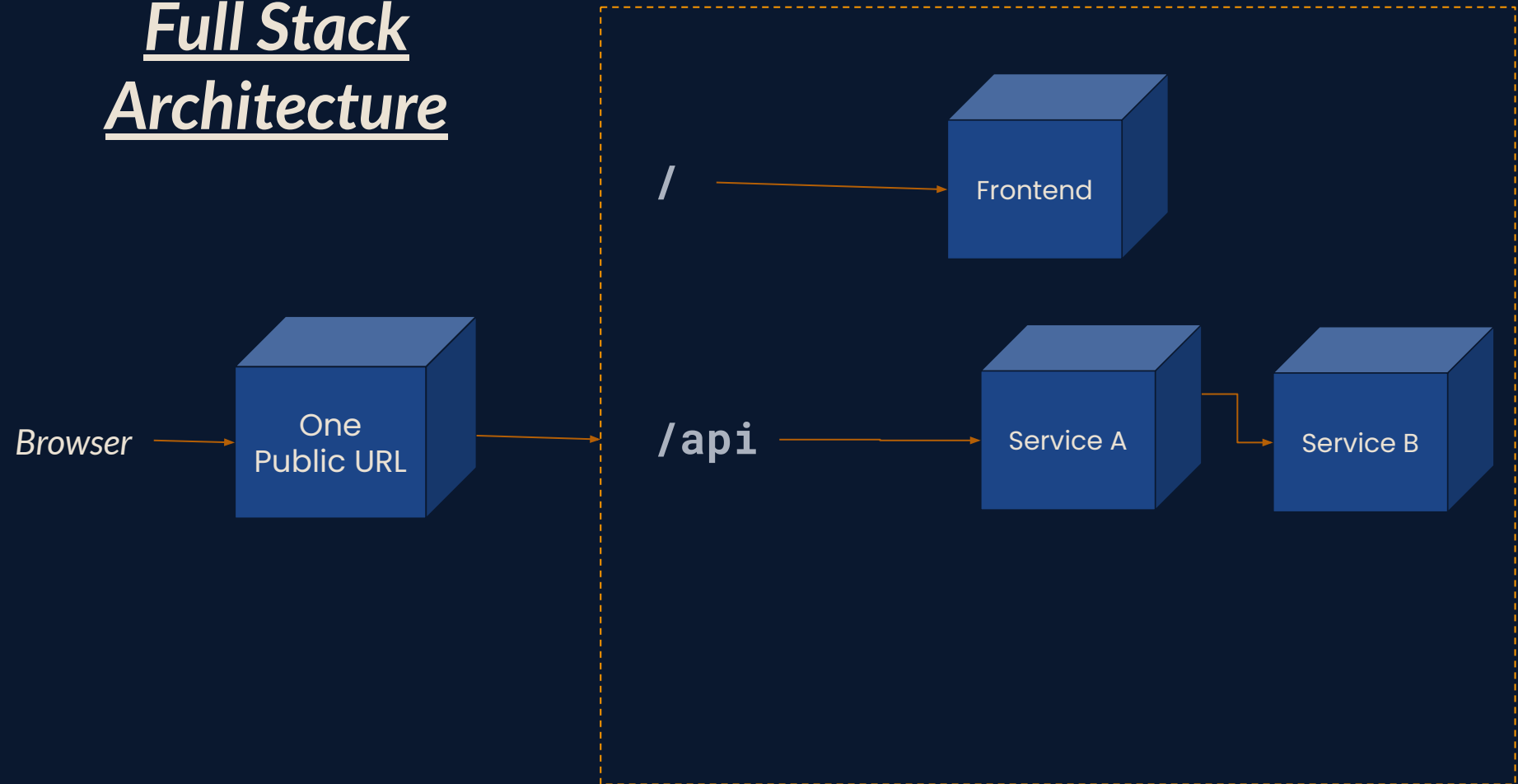
*1 cluster → many nodes → many pods*

# Understanding the Full Stack Architecture with Microservices

---

Faisal Memon (EmbarkX)

# Full Stack Architecture



# Ingress

---

Faisal Memon (EmbarkX)

Ingress in Kubernetes is used to  
**expose multiple services** to  
external users in a clean, controlled  
way using a **single entry point**.

# Ingress

## **Without Ingress**

- Every service needs its own LoadBalancer / NodePort
- Expensive (cloud LB per service)
- Hard to manage URLs

## **With Ingress**

One Load Balancer

Smart HTTP/HTTPS routing to many services

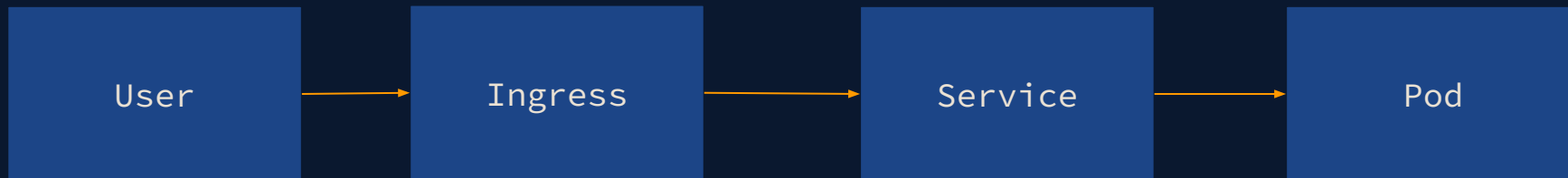
Central place for TLS, auth, rate-limits



## *What Ingress gives you*

- *Single public endpoint*
- *Path / host-based routing*
- *Lower cloud cost*

# *Request Flow with Ingress*



## *When you should use Ingress*

- *Microservices*
- *Single domain, many APIs*
- *Production setups*
- *Need HTTPS + routing*

## *When you don't need it*

- *Single service*
- *Local learning / quick demos*

**Ingress** = HTTP/HTTPS traffic manager for Kubernetes, sitting in front of multiple services.

# Ingress Controller

---

Faisal Memon (EmbarkX)

**Ingress Controller** is the actual running software that reads Ingress rules and handles real HTTP traffic.

# Ingress Controller is...

- *A normal application*
- *Running inside Kubernetes*
- *Running as Pods*
- *Is managed by a Deployment*
- *Exposed via a Service*

# *What runs inside those Pods*

A web  
traffic  
engine

Controller  
logic



Kubernetes **doesn't ship one** by  
default

# Understanding Ingress Configurations

---

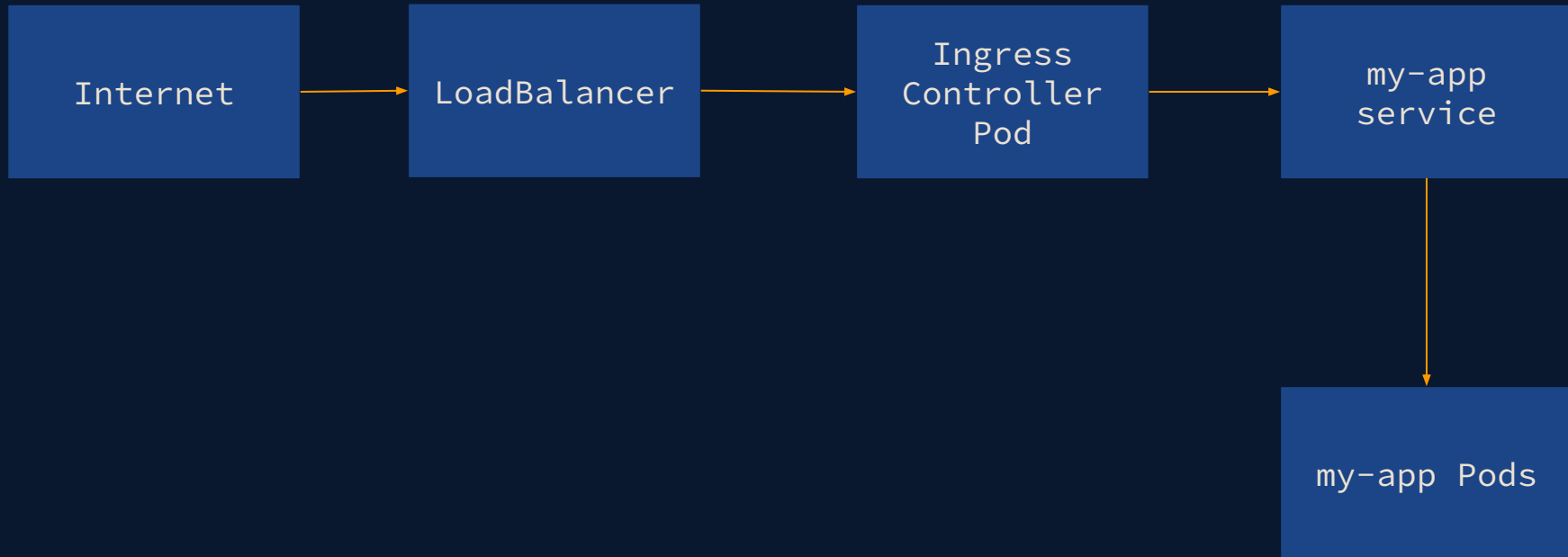
Faisal Memon (EmbarkX)

# Ingress Configuration

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: simple-ingress
spec:
 ingressClassName: nginx

 rules:
 - http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: my-app
 port:
 number: 80
```

# How Traffic Flows



```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: multi-service-ingress
spec:
 ingressClassName: nginx

 rules:
 - http:
 paths:
 # Backend API
 - path: /api
 pathType: Prefix
 backend:
 service:
 name: backend
 port:
 number: 80

 # Frontend
 - path: /
 pathType: Prefix
 backend:
 service:
 name: frontend
 port:
 number: 80
```

# Ingress with Multiple Services

*Example:*

*GET /api/users → backend receives /api/users*

*GET /about → frontend receives /about*

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: rewrite-ingress
 annotations:
 # REQUIRED for regex paths
 nginx.ingress.kubernetes.io/use-regex: "true"

 # Rewrite path using capture groups
 nginx.ingress.kubernetes.io/rewrite-target: /$1
spec:
 ingressClassName: nginx

 rules:
 - http:
 paths:
 - path: /api/(.*)
 pathType: Prefix
 backend:
 service:
 name: backend
 port:
 number: 80

 - path: /(.*)
 pathType: Prefix
 backend:
 service:
 name: frontend
 port:
 number: 80
```

# Ingress with Multiple Services, Regex & Rewriting Rules

## EXAMPLE:

1. *Receives: GET /api/users*
2. *Checks rules (top to bottom)*
3. *Matches: /api/(.\*)*
4. *Captures: "users" in \$1*
5. *Applies rewrite-target: /\$1*
6. *Transforms to: GET /users*
7. *Forwards to: service-a:80*

## Example Requests

| User Request    | Matched Rule | Captured (\$1) | Rewritten To   | Service   |
|-----------------|--------------|----------------|----------------|-----------|
| /api/users      | /api/(.*)    | users          | /users         | service-a |
| /api/products/5 | /api/(.*)    | products/5     | /products/5    | service-a |
| /api/           | /api/(.*)    | (empty)        | /              | service-a |
| /about          | /(.*)        | about          | /about         | frontend  |
| /               | /(.*)        | (empty)        | /              | frontend  |
| /static/app.js  | /(.*)        | static/app.js  | /static/app.js | frontend  |

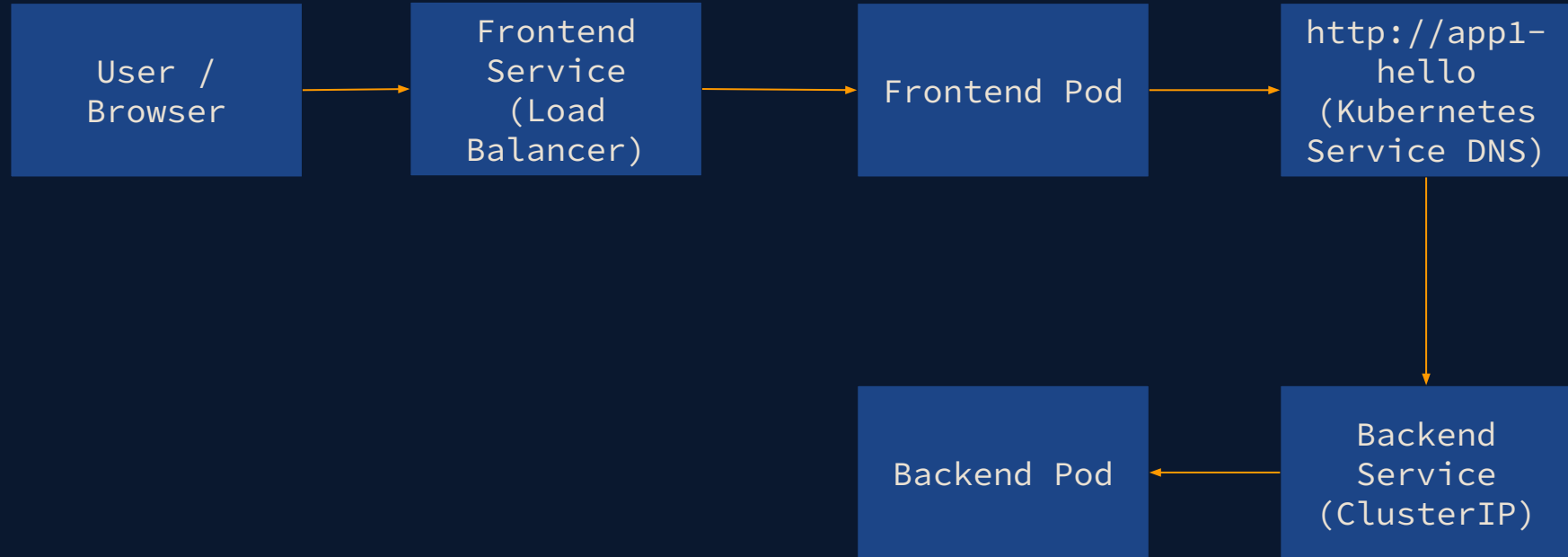
# Deploying Full Stack Application on AWS EKS

---

Faisal Memon (EmbarkX)



# *Request Flow in AWS EKS*

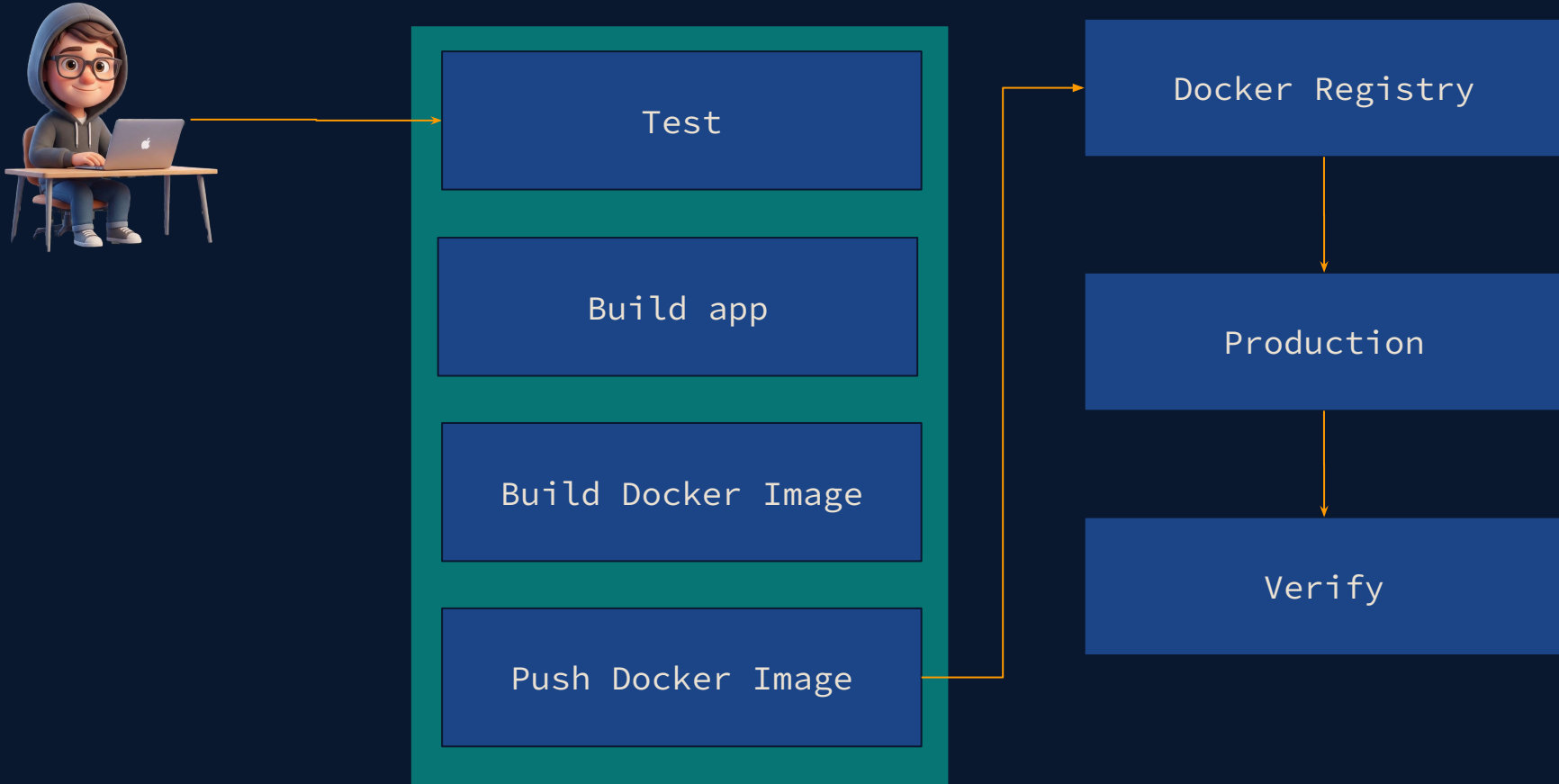


# Before CI/CD

---

Faisal Memon (EmbarkX)

# Before CI/CD



# Software Development Pipeline



# *Before CI/CD*

→ *Manual Workflow*

→ *Human Dependency & Risk*

→ *Slow & Painful Feedback Loop*

Should growing team  
**trust humans** every  
time?

# Continuous Integration and Continuous Deployment (CI/CD)

---

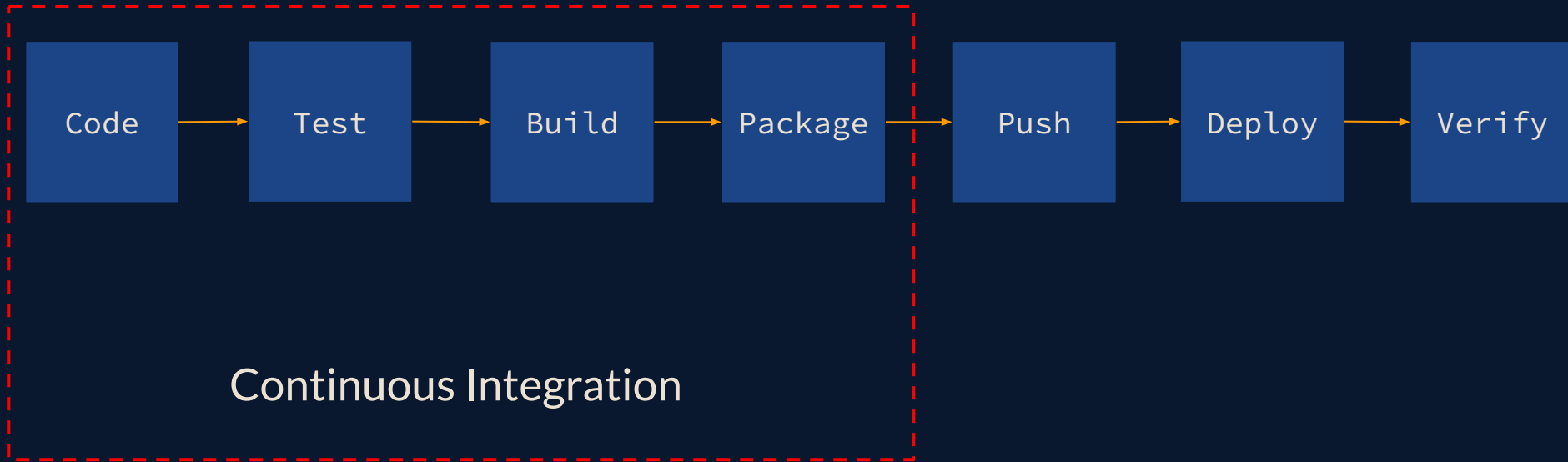
Faisal Memon (EmbarkX)

# Software Development Pipeline



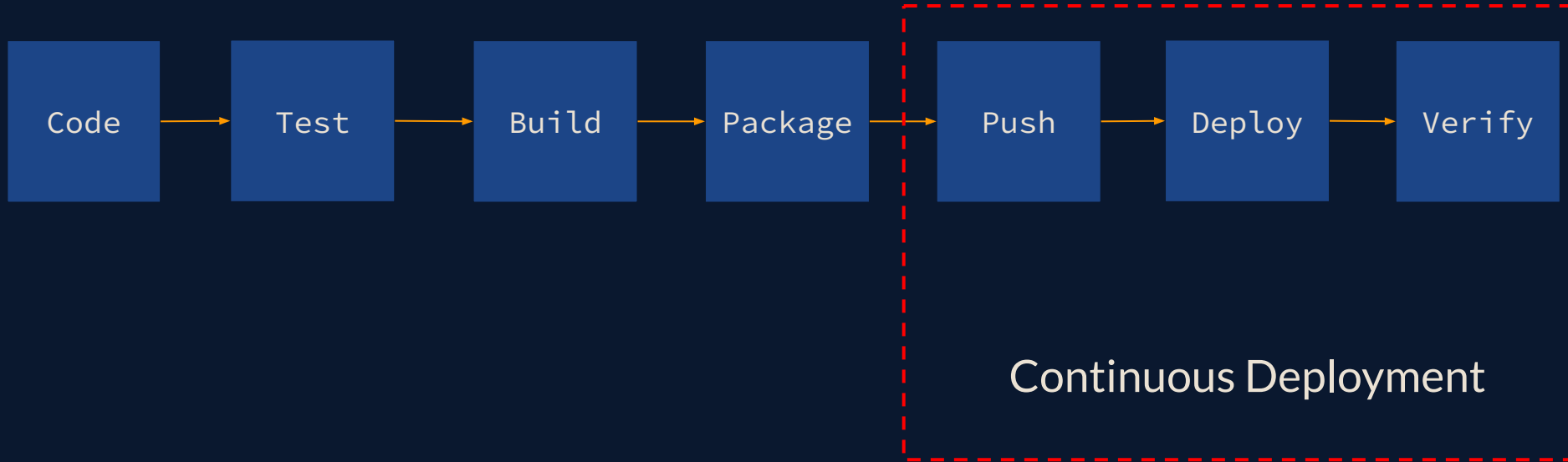


# Software Development Pipeline



**Continuous Integration**  $\Rightarrow$  Every code change is automatically built and tested as soon as it is pushed to the repository.

# Software Development Pipeline



Continuous Integration  $\Rightarrow$  Every successful build is automatically deployed to an environment without manual intervention.

# Manual vs CI/CD

*In pre-CI/CD era, all these steps were:*

- Manually triggered*
- Inconsistent*
- Error-prone*

*This created:*

- Broken releases*
  - Late feedback*
  - Fear of deployments*
- CI/CD was created to automate this entire flow*

Before CI/CD, software quality  
depended on human discipline,  
**not automation**

# Understanding CI/CD Pipelines & Pipeline Engines

---

Faisal Memon (EmbarkX)

A **pipeline** is a predefined sequence of steps that run automatically to take your code from developer to production.

**Pipeline** = Steps +  
Order + Automation

**Jenkins & GitHub Actions** are  
tools that run pipelines.



# What Does a Pipeline Look Like?

```
pipeline:
```

```
 trigger: code_push
```

```
 stages:
```

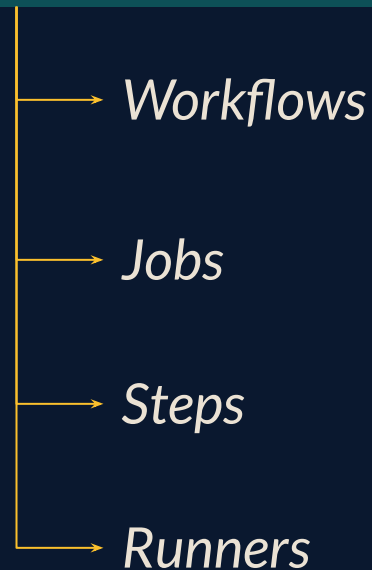
- test
- build
- docker
- deploy
- verify

# CI/CD Fundamentals & Terminology

---

Faisal Memon (EmbarkX)

# CI/CD Fundamentals & Terminology

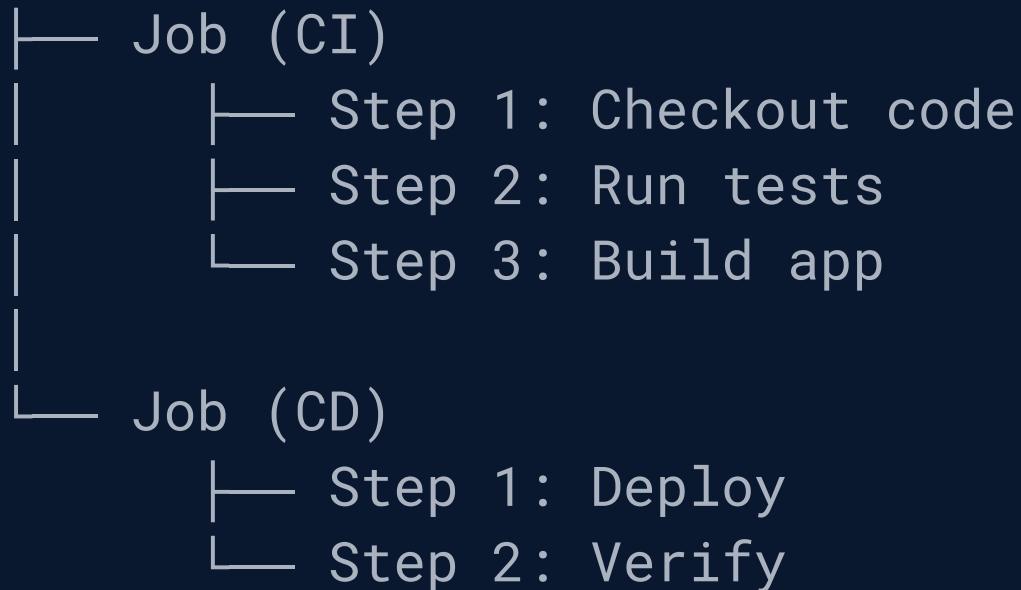


# CI/CD Fundamentals & Terminology

- *A workflow is the full automation process.*
- *A job is a major task inside a workflow.*
- *Steps are the individual commands inside a job.*
- *A runner is the computer where your job runs.*

# How They Fit Together

## Workflow



## *Build & Test*

Step 1: Checkout code  
Step 2: Run tests  
Step 3: Build app

## *Build Image & Deploy*

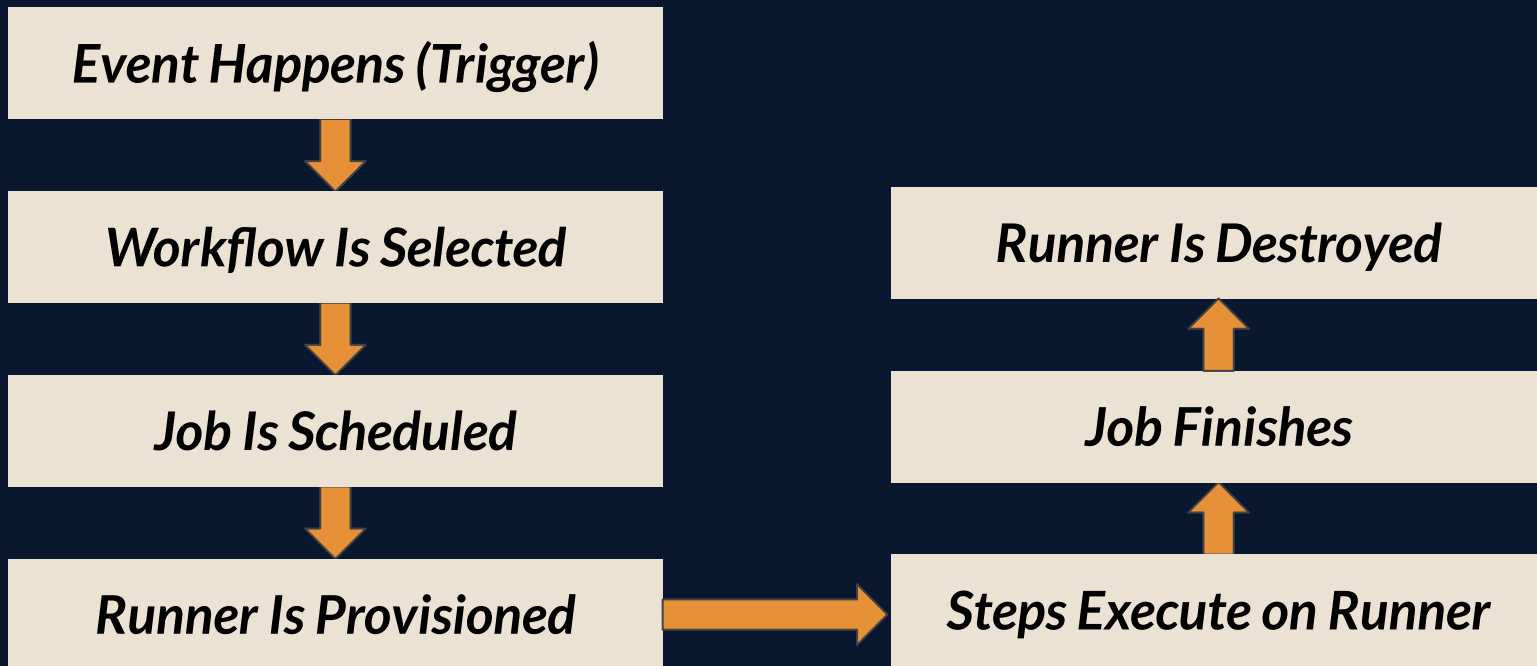
Step 1: Build Docker Image  
Step 2: Push Docker Image  
Step 3: Deploy to Production

*STEPS*

*JOB*

*WORKFLOW*

# What Really Happens?



# Anatomy of a CI Pipeline

---

Faisal Memon (EmbarkX)



# Anatomy of CI Pipeline

```
name: Demo CI
```

```
on:
 push
```

```
jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - name: Say Hello
 run: echo "Hello from the second workflow!"
```

## *name* → *Identity*

```
name: Demo CI
```

```
on:
 push
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
 steps:
 - name: Say Hello
 run: echo "Hello from the second workflow!"
```

## on → *Trigger*

name: Demo CI

```
on:
 push
```

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Say Hello

run: echo "Hello from the second workflow!"

## jobs → *Work*

```
name: Demo CI
```

```
on:
 push
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
 steps:
 - name: Say Hello
 run: echo "Hello from the second workflow!"
```

## steps → Actions

```
name: Demo CI
```

```
on:
 push
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
 steps:
 - name: Say Hello
 run: echo "Hello from the second workflow!"
```

## run / uses → Execution

```
name: Demo CI
```

```
on:
 push
```

```
jobs:
 build:
 runs-on: ubuntu-latest
```

```
 steps:
 - name: Say Hello
 run: echo "Hello from the second workflow!"
```

## *Different Syntax, Same Meaning*

```
on:
 push
```

```
on:
 push:
 pull_request:
```

```
on:
 push:
 branches: ["working"]
 pull_request:
 branches: ["working"]
```

# Running Commands vs Using Actions

---

Faisal Memon (EmbarkX)



*Every single step does only ONE of two things*



*Run a command*

*Use a pre-built action*

# Running a command

- **name:** Say Hello

**run:** echo "Hello from CI"

*Create a Linux machine and run this command in its terminal.*

→ *run executes shell commands*

→ *Same commands you run locally*

→ *Same syntax as Linux terminal*

**run:** mvn test

**run:** npm install

**run:** docker build .

**run:** python app.py

# Use a pre-built action

- `name:` Checkout Code

`uses:` actions/checkout@v4

*Run a pre-built automation written by someone else.*

→ *You are NOT writing commands here.*

→ *You are reusing logic.*

# What are these “pre-defined actions”?

→ *Pre-defined actions = reusable automation packages*

→ *They are written by GitHub or the community, versioned (@v4), stored in repositories, executed for you*

```
actions/checkout@v4 # download your repo code
actions/setup-java@v4 # install Java
docker/login-action@v3 # login to Docker registry
actions/upload-artifact@v4
```

# Why do we even need uses? Why not just run?

*Let's take checkout as an example...*

```
run: git clone https://github.com/my/repo.git
```

*So github gives you:*

```
uses: actions/checkout@v4
```

# *The runner has NO CODE by default*

→ *GitHub creates a brand-new machine. That machine has no project code*

- `uses: actions/checkout@v4`
- `run: ls`

## “run” and “uses” cannot be used together

*Below is invalid*

- `name: Bad Step`  
  `run: echo "Hello"`  
  `uses: actions/checkout@v4`

*A step can only do one thing: Either run commands Or execute an action*

*Below is valid*

- `steps:`
  - `uses: actions/checkout@v4`
  - `name: Build project`  
  `run: mvn clean package`

## “run” vs. “use”

### Use run when:

- The command is specific to your project
- You control the logic
- It's simple and transparent

### Use uses when:

- Task is common
- Security matters
- Setup is complex
- Best practices already exist

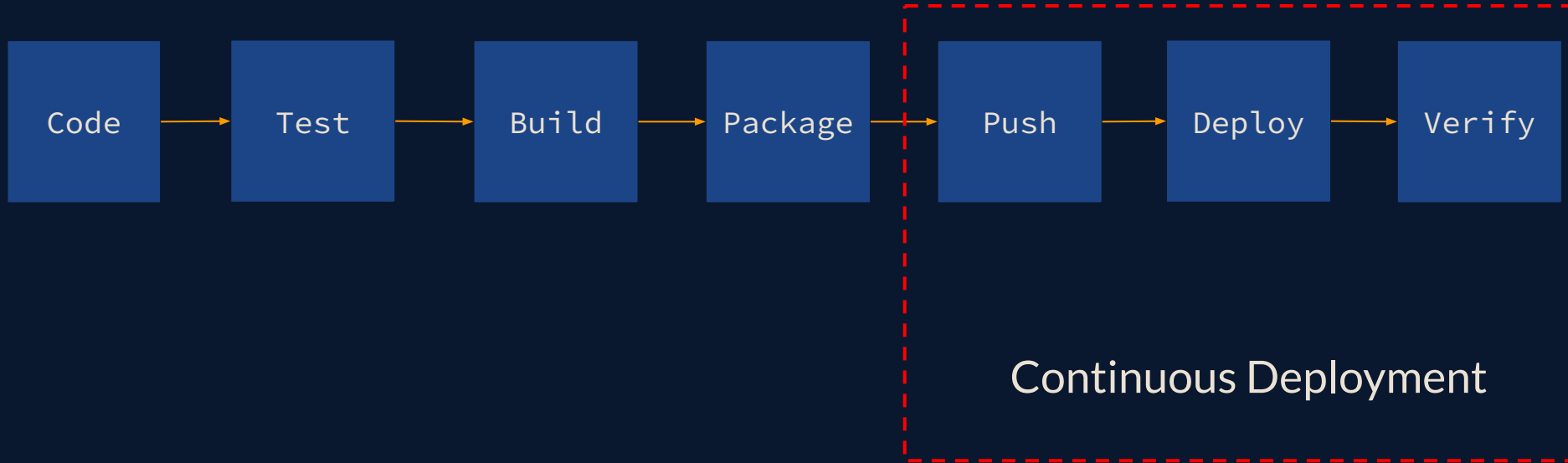


# Continuous Deployment (CD)

---

Faisal Memon (EmbarkX)

# Software Development Pipeline



Continuous Deployment  $\Rightarrow$  Every successful build is automatically deployed to an environment without manual intervention.

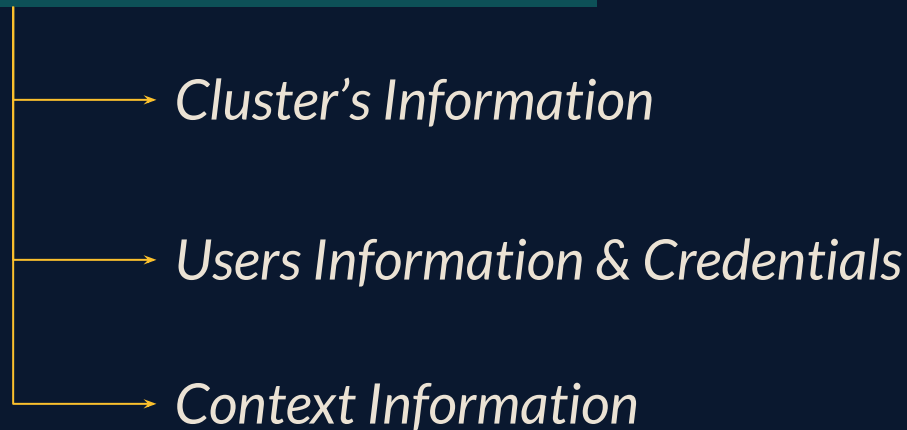
# How CI/CD Talks to Kubernetes

---

Faisal Memon (EmbarkX)

**kubeconfig** is a **config file** that tells *kubectl* **how to connect** to a Kubernetes cluster.

# *What's inside a kubeconfig?*



# Why do we use kubeconfig?

- *Switching Environments*
- *Automation*
- *Security*
- *Ability to connect to clusters information*

# Commands

```
kubectl config view
```

```
kubectl config current-context
```

```
kubectl config use-context <name>
```

## *Default location*

*Linux / macOS: ~/.kube/config*

*Windows: %USERPROFILE%\kube\config*



# When is kubeconfig created?

## **Docker Desktop (local Kubernetes)**

*Automatically when you enable Kubernetes in Docker Desktop.*

## **AWS (EKS)**

*When you create cluster, automatically - behind the scenes*

# What happens inside GitHub Actions

→ *A brand new VM*

→ *No memory*

→ *No files*

→ *No kubeconfig*

# Create kubeconfig for CI/CD

```
aws eks update-kubeconfig --name <CLUSTER-NAME> --region ap-south-1
```

# Infrastructure as a Code

---

Faisal Memon (EmbarkX)

You define your **infrastructure**  
**using code** instead of **manually**  
**clicking** in cloud dashboards.

## Without IaC

- *Someone clicks around AWS/GCP console*
  - *Creates VMs, networks, rules manually*
- *Infra is written in files (like main.tf)*
  - *Stored in Git*

## Example Server

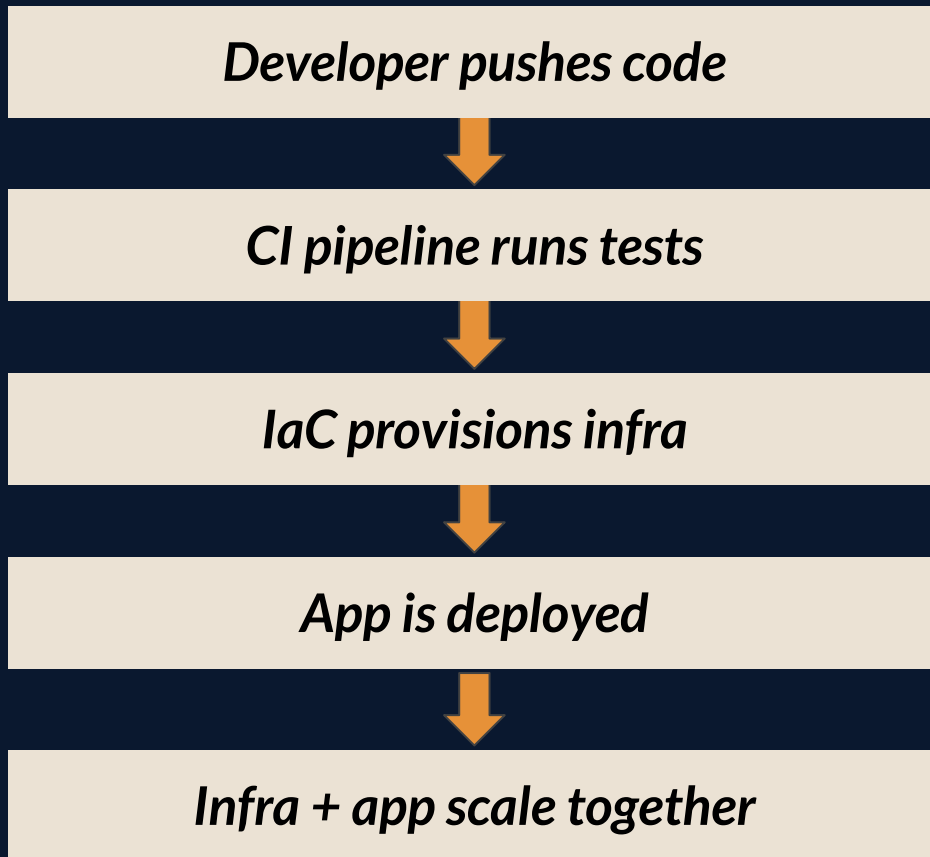
```
resource "aws_instance" "app" {
 ami = "ami-0abc123"
 instance_type = "t3.micro"
}
```

# Why IaC Matters

- *Consistency*
- *Speed & Automation*
- *Version Control for Infrastructure*
- *Safe Collaboration (Teams)*
- *Reduced Human Errors & Easy to Scale*



## *Where IaC Fits in DevOps*

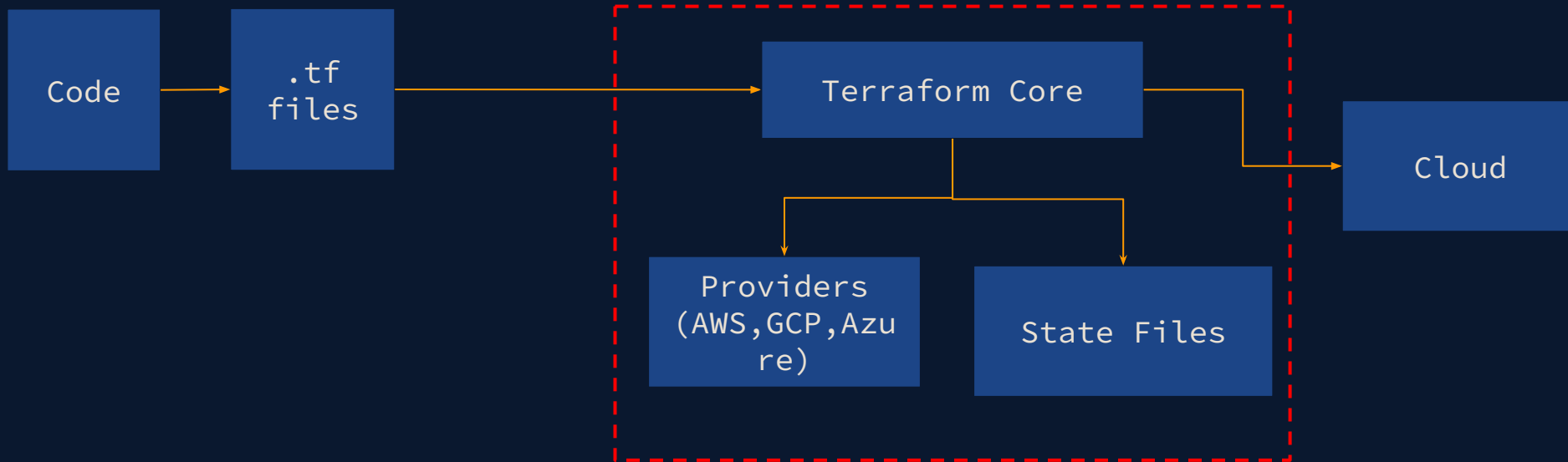


# What is Terraform?

---

Faisal Memon (EmbarkX)

# How Terraform Works?

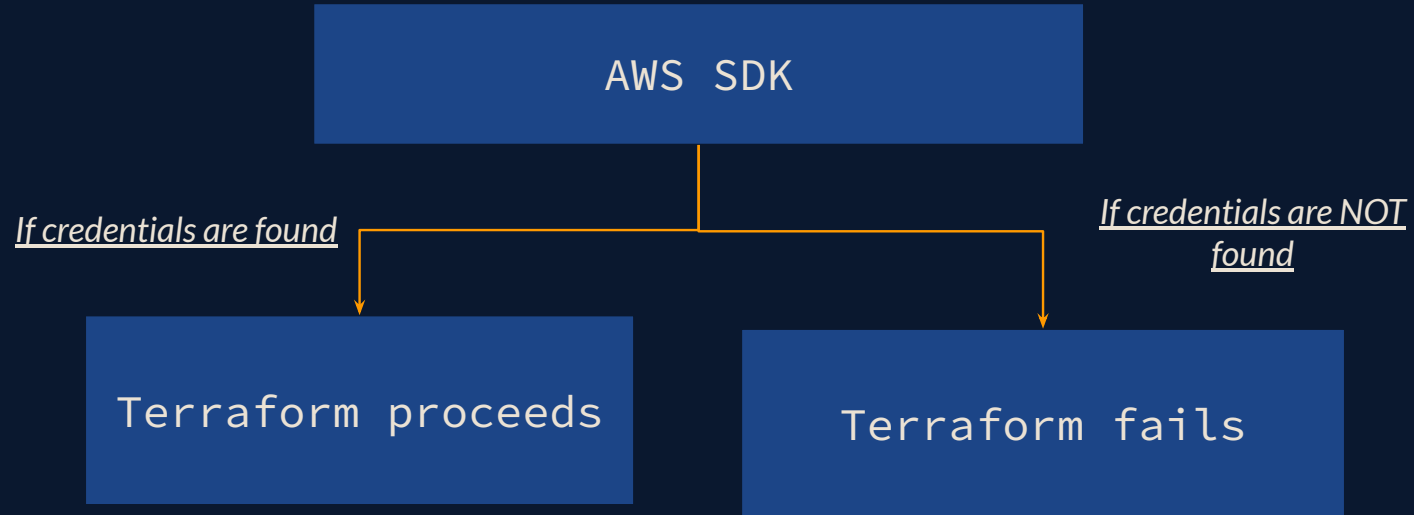


# Talking to AWS For Creating Resources

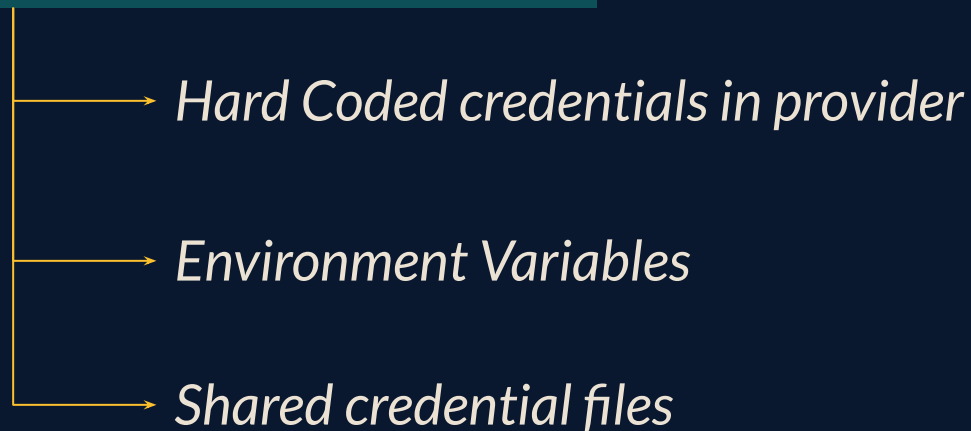
---

Faisal Memon (EmbarkX)

Terraform **never prompts** for  
credentials because it is  
designed for **automation**



# Credential Resolution

- 
- ```
graph LR; A[Credential Resolution] --> B[Hard Coded credentials in provider]; A --> C[Environment Variables]; A --> D[Shared credential files];
```
- *Hard Coded credentials in provider*
 - *Environment Variables*
 - *Shared credential files*

Credentials in Provider

```
provider "aws" {  
  region      = "us-west-2"  
  access_key  = "AKIAEXAMPLE12345"  
  secret_key  =  
  "wJa1rXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"  
}
```


Environment Variables

```
export AWS_ACCESS_KEY_ID = XXXXX
```

```
export AWS_SECRET_ACCESS_KEY = XXXXX
```

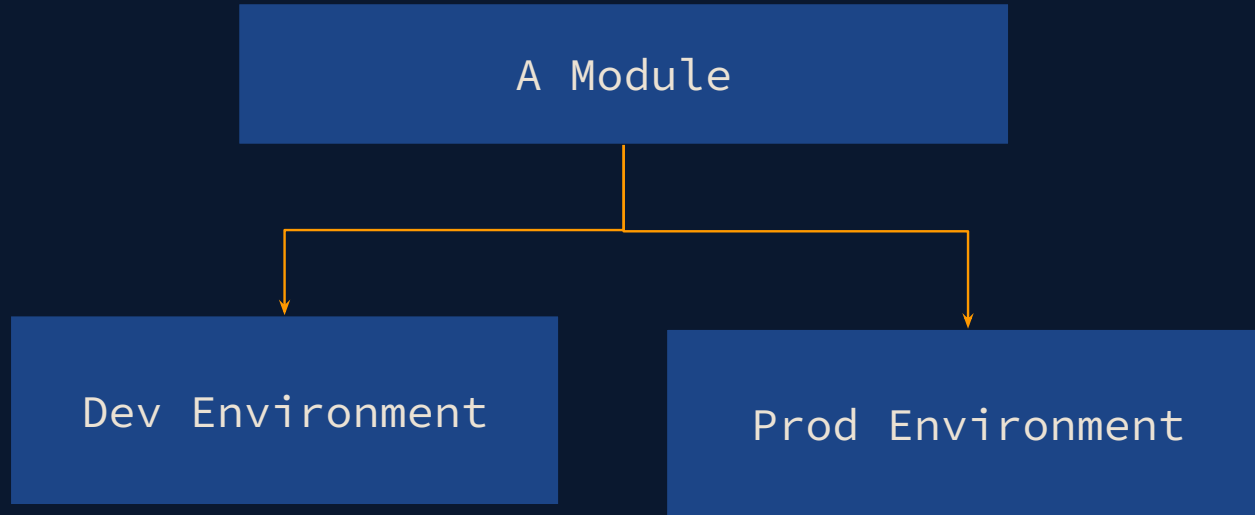
Shared Credentials Files

- *If environment variables are not set, Terraform looks for the shared credentials file.*
- *The default location is “~/.aws/credentials” on Linux/macOS, or “%USERPROFILE%\aws\credentials” on Windows.*
- *When you run “aws configure”. Terraform sets above location*
- *Run “aws configure list” to see credentials on your machine*

Terraform Modules

Faisal Memon (EmbarkX)

A separate Terraform
configuration that can be
**reused, versioned, and
composed.**



Different Environments

Environment	Instance	Monitoring
dev	t2.micro	disabled
test	t2.small	enabled
prod	t3.medium	enabled

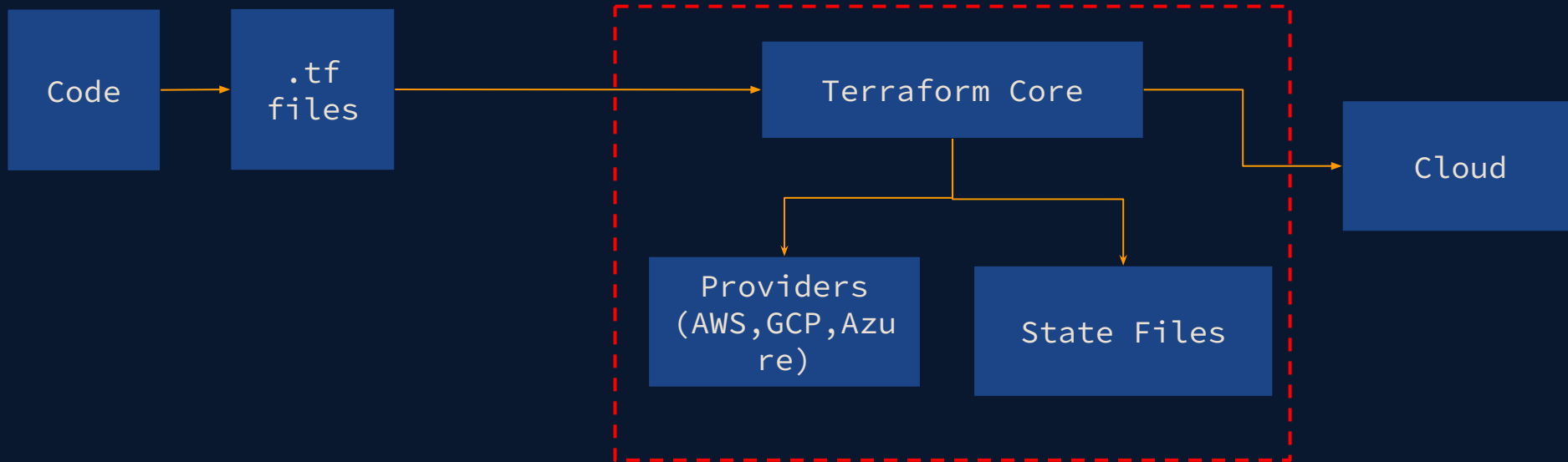
Challenge

Environment	Instance	Monitoring	Versioning
dev	t2.micro	disabled	disabled
test	t2.small	enabled	enabled
prod	t3.medium	enabled	enabled

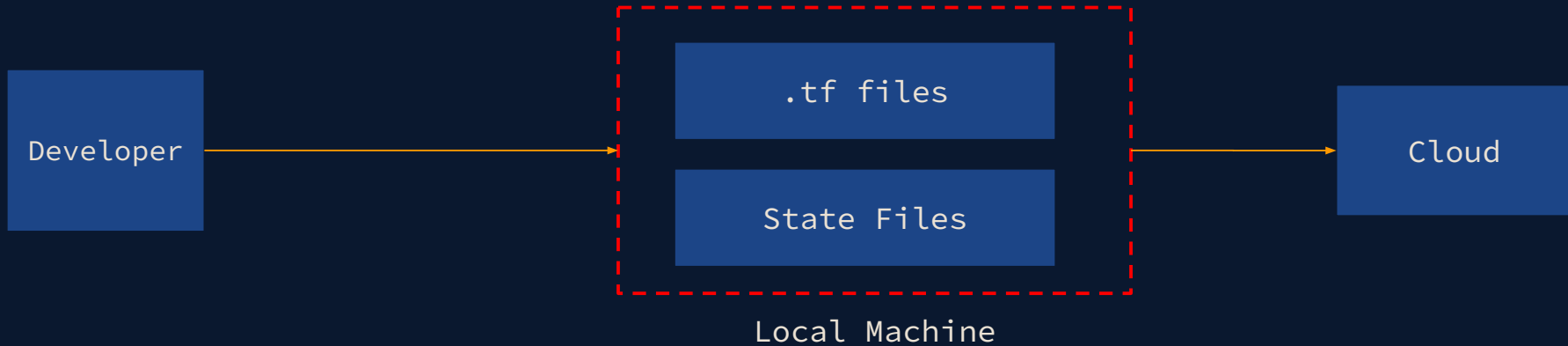
Remote States in Terraform

Faisal Memon (EmbarkX)

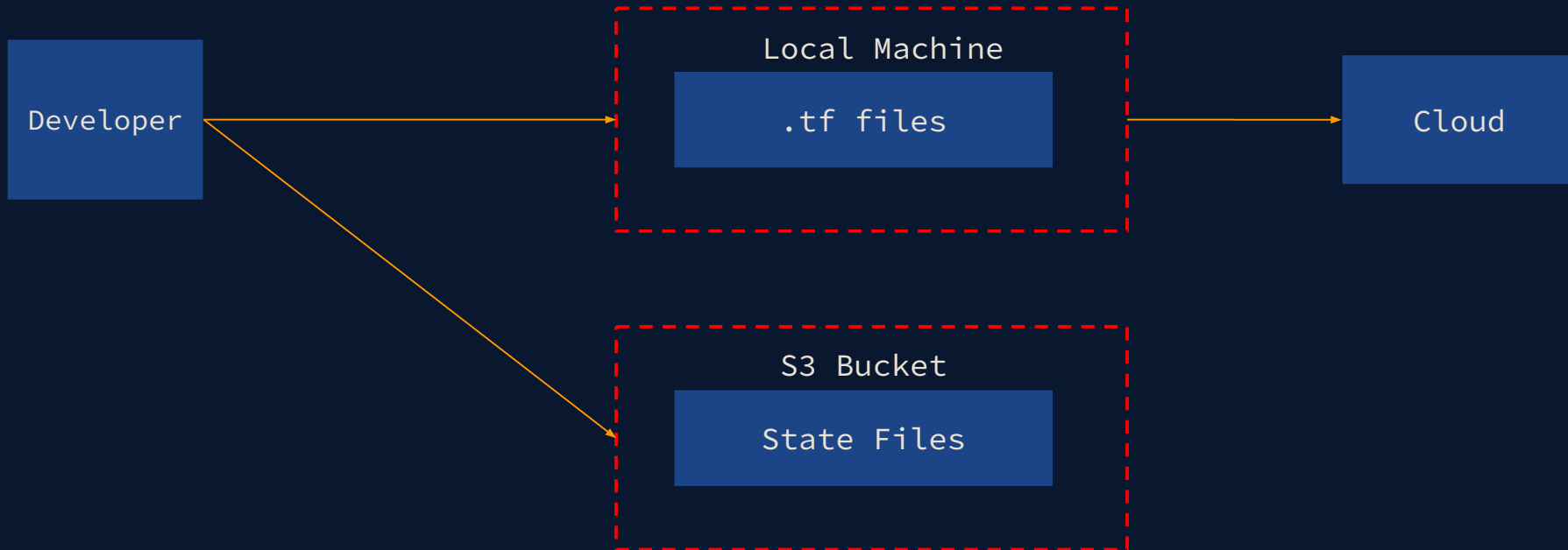
How Terraform Works?



Terraform States on Local Machine



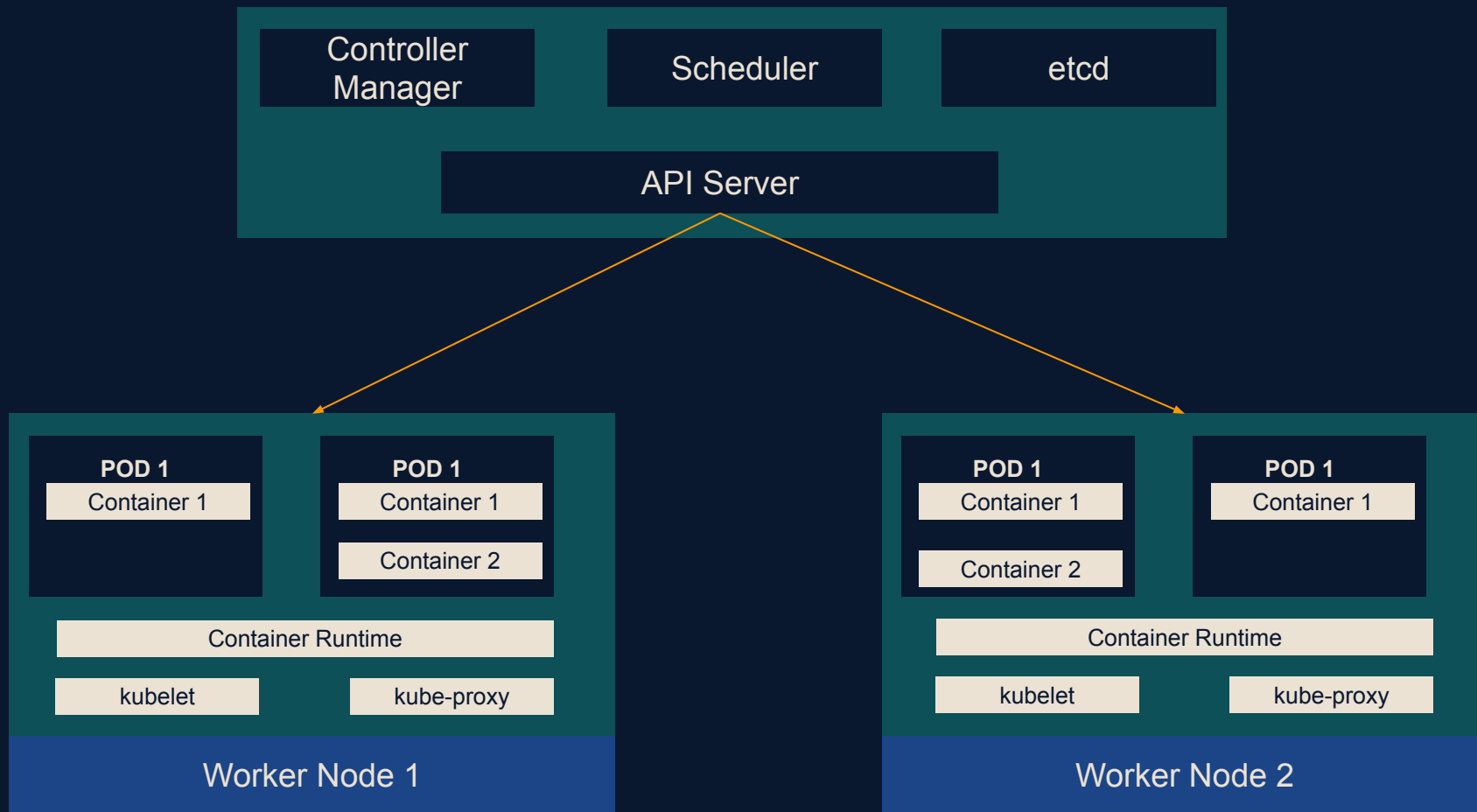
Remote State Management

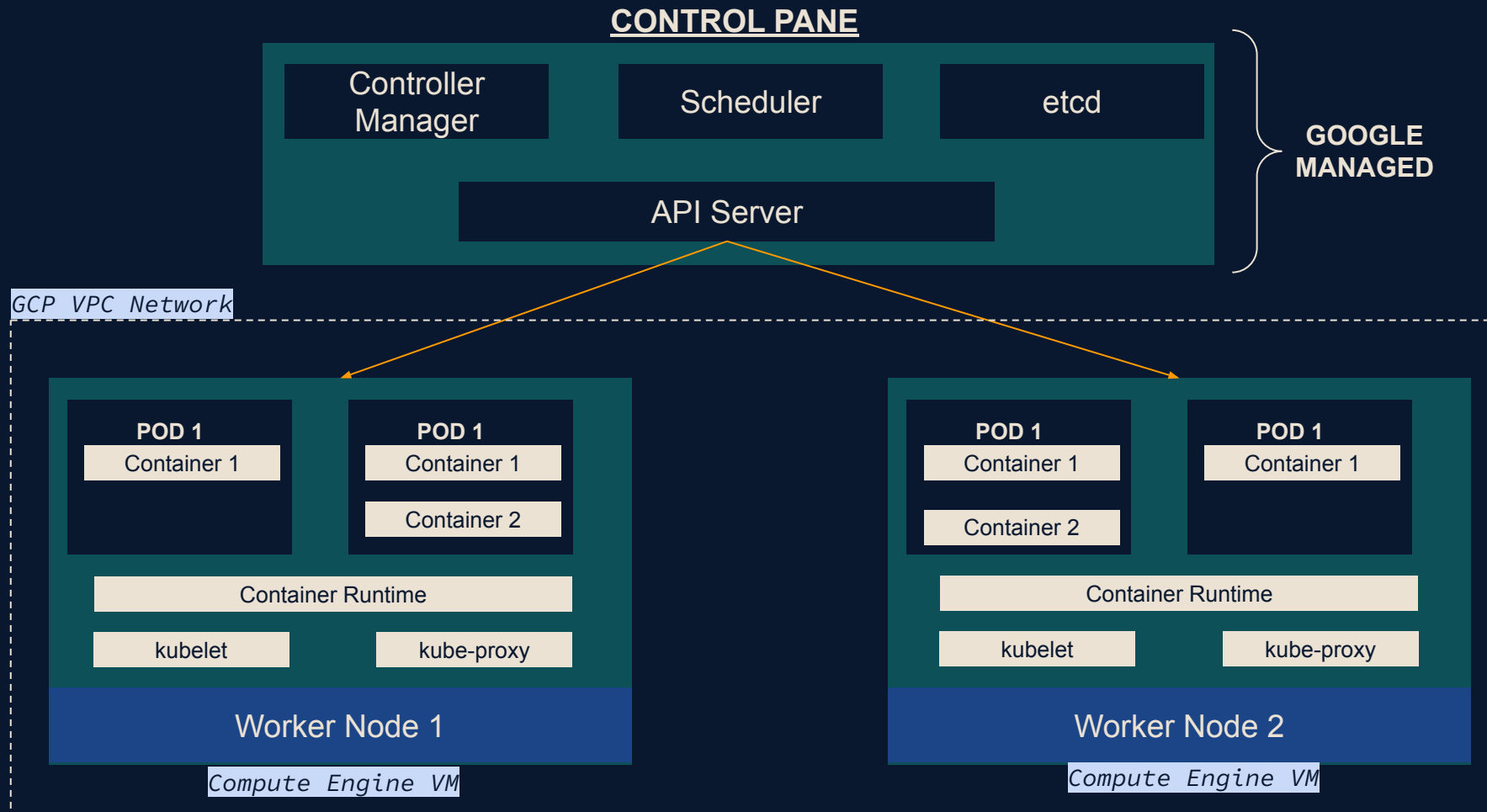


GKE Architecture

Faisal Memon (EmbarkX)

CONTROL PANE





Instructor: Faisal Memon

Company: EmbarkX.com

1. Personal Use Only

The materials provided in this course, including but not limited to PDF presentations, are intended for your personal use only. They are to be used solely for the purpose of learning and completing this course.

2. No Unauthorized Sharing or Distribution

You are not permitted to share, distribute, or publicly post any course materials on any websites, social media platforms, or other public forums without prior written consent from the instructor.

3. Intellectual Property

All course materials are protected by copyright laws and are the intellectual property of Faisal Memon and EmbarkX. Unauthorized use, reproduction, or distribution of these materials is strictly prohibited.

4. Reporting Violations

If you become aware of any unauthorized sharing or distribution of course materials, please report it immediately to [\[embarkxofficial@gmail.com\]](mailto:embarkxofficial@gmail.com).

5. Legal Action

We reserve the right to take legal action against individuals or entities found to be violating this usage policy.

Thank you for respecting these guidelines and helping us maintain the integrity of our course materials.

Contact Information

embarkxofficial@gmail.com

www.embarkx.com

If you think this course helped you, please do help provide an **honest rating and review** of the course. Your insights help us **improve and provide better content** for future learners.

We appreciate your **support** and look forward to hearing your **thoughts!**

The Ultimate Java and Spring Boot Mastery

Welcome to your one-stop-shop for mastering Java and Spring Boot! This repository offers a comprehensive learning experience with high-quality resources and community support. Dive into over 150+ hours of premium content, with everything you need to excel at Java and Spring Boot development.

Learning Roadmap

Most of the courses below are available in Udemy For Business, so if you have subscription - you can get FREE access. Here's a structured path to enhance your skills with detailed courses available:

1. [Spring Boot Full Stack By Building eCommerce Application Step by Step](#) (90+ Hours of Content)
2. [Master Spring Boot Microservices](#) (70+ Hours of Content)
3. [Learn Java with 60+ Hours of Content](#) (60+ Hours of Content)
4. [Master Spring Security with React JS + OAuth2](#) (34+ Hours of Content)
5. [Master IntelliJ IDEA](#) (3+ Hours of Content)

With All Our Courses You Gain Access To

-  Notes: Detailed and downloadable notes to accompany each lesson.
-  Source Code: Full access to the source code used in the tutorials.
-  Doubt Solving: Responsive instructor and community support.
-  High-Quality HD Videos: Easy to understand, high-definition video tutorials.
-  Free Lifetime Updates: Continuous updates to course content at no extra cost.